다차원 다중분류와 Softmax, Cross Entropy loss

Affine → Activation → Softmax → Cross Entropy → Mini-batch → MNIST

이번 주의 핵심 질문

지난주에는 입력 x 하나, 출력 $\hat{y}$ 하나인 network를 MSE로 학습하였다. 그러나 실제 문제에서는

- 입력이 **여러 개의 변수**로 이루어진다. (예: 28×28 손글씨 이미지 $\Rightarrow$ 784개의 픽셀 값)
- 정답이 **여러 개의 class** 중 하나이다. (예: 숫자 $0, 1, \ldots, 9 \Rightarrow$ 10개의 class)

그렇다면 network의 출력을 어떻게 "각 class일 확률"로 바꾸고, 어떤 손실함수로 학습하며, 행렬로 된 parameter의 gradient는 어떻게 계산할까? 그리고 60000개나 되는 데이터를 어떻게 효율적으로 학습시킬까?

학습목표

이 주차가 끝나면 다음을 설명하고 직접 계산할 수 있어야 한다.

1. 다차원 다중분류 network의 각 변수와 parameter의 shape을 말할 수 있다.
2. affine 계층 $A = XW + B$의 역전파 $\frac{\partial l}{\partial X}$, $\frac{\partial l}{\partial W}$, $\frac{\partial l}{\partial B}$를 성분 계산으로부터 유도한다.
3. sigmoid와 ReLU 활성화 계층의 역전파를 성분별 곱으로 쓸 수 있다.
4. 정답을 one-hot vector로 나타내고, 출력 점수를 softmax로 확률분포로 바꾸는 이유를 설명한다.
5. 두 확률분포를 비교하는 cross entropy 손실함수를 이해한다.
6. softmax와 cross entropy를 묶은 계층의 역전파가 $Y - t$임을 유도한다.
7. mini-batch 학습에서 평균 손실의 gradient를 행렬곱과 `np.sum`으로 계산한다.
8. NumPy만으로 MNIST 손글씨 숫자 분류 network를 구현하고 학습시킨다.

1 다차원 다중분류 문제와 신경망의 구조

1.1 문제 설정

l차원의 입력 변수

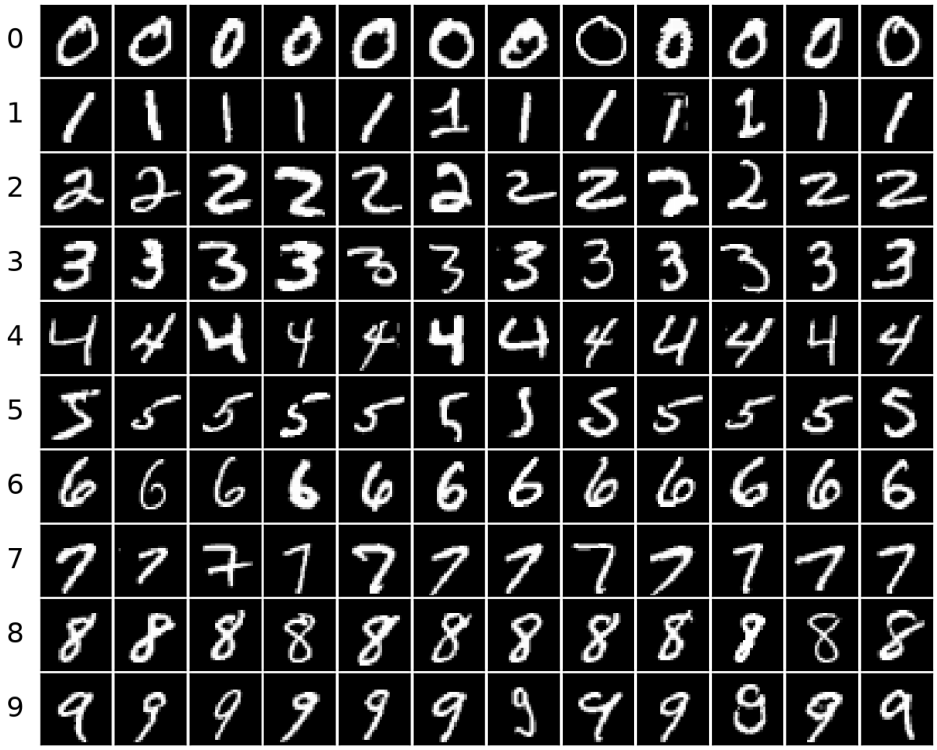
$$X_{1\times l} = (x_1, x_2, \ldots, x_l)$$

를 n개의 class $\{1, 2, \ldots, n\}$ 중 하나로 분류하는 문제를 생각하자.

- 입력 변수의 차원이 $l > 1$인 경우를 **다차원** 문제라 한다.

- class의 수가 $n = 2$이면 **이진분류(binary classification)**, $n > 2$이면 **다중분류(multi-class classification)**라 한다.

이번 주에 다룰 다차원 다중분류의 대표적인 예가 **MNIST 손글씨 숫자 데이터**이다. MNIST는 사람들이 손으로 쓴 숫자 $0 \sim 9$의 흑백 이미지 70000장(훈련용 60000장, test용 10000장)으로 이루어져 있다.

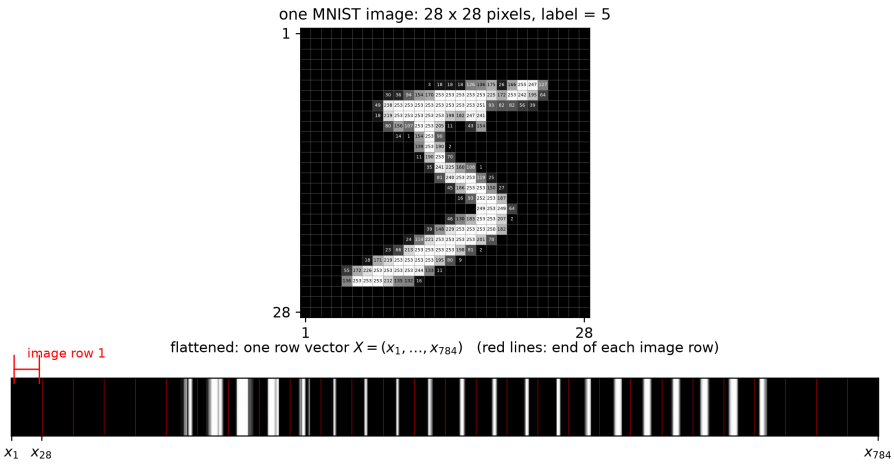


MNIST 훈련 데이터의 예. 각 행은 class $0, 1, \ldots, 9$이며, 같은 숫자도 쓰는 사람에 따라 모양이 매우 다양하다.

각 이미지는 $28 \times 28 = 784$개의 픽셀로 되어 있고, 각 픽셀은 밝기를 나타내는 0(검정)$\sim$ 255(흰색)의 정수이다. 이 784개의 픽셀 값을 첫 번째 줄부터 차례로 **한 줄로 늘어놓으면** 하나의 행벡터

$$X_{1\times 784} = (x_1, x_2, \ldots, x_{784})$$

가 되고, 이것이 network에 들어가는 **하나의 훈련 데이터(training example)**이다. 각 이미지의 첫 번째 줄이 $x_1 \sim x_{28}$, 두 번째 줄이 $x_{29} \sim x_{56}$, ..., 마지막 줄이 $x_{757} \sim x_{784}$가 된다.



하나의 MNIST 이미지(정답 label = 5)와 그 픽셀 값(위), 그리고 이를 한 줄로 펼친 1×784 행벡터(아래). 빨간 선은 이미지의 각 줄이 끝나는 위치이다.

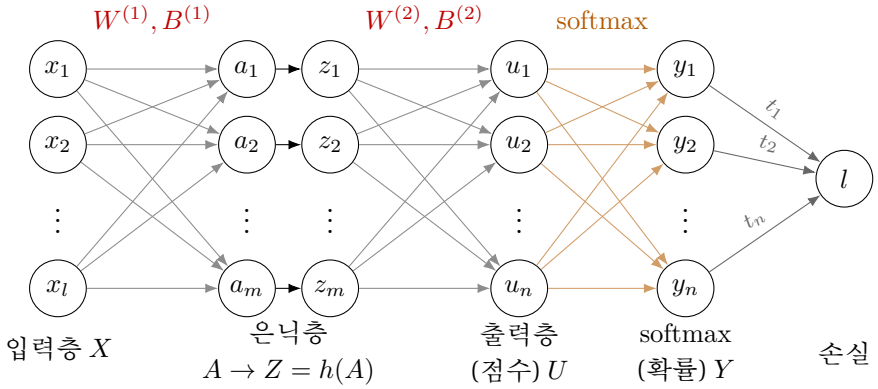
정답(label)은 이미지가 나타내는 숫자이므로 class는

$$\{0, 1, 2, \ldots, 9\} \qquad (n = 10)$$

의 10개이다. 즉 MNIST 분류는 $l = 784$**차원의 입력을** $n = 10$**개의 class 중 하나로 분류하는 다차원 다중분류 문제**이다. ($0, \ldots, 9$의 label을 class $1, \ldots, 10$으로 번호만 바꿔 생각하면 된다.)

1.2 신경망의 구조

은닉층 하나와 softmax 출력층으로 구성된 다음 network를 생각하자.



입력층의 변수를 $X_{1\times l}$, 은닉층의 변수를 $A_{1\times m}$, A에 활성화 함수 h를 적용한 변수를 $Z_{1\times m}$, 출력층의 변수를 $U_{1\times n}$, softmax 함수의 출력을 $Y_{1\times n}$이라 하자. 이 변수들은 가중치 행렬 $W^{(1)}_{l\times m}$, $B^{(1)}_{1\times m}$, $W^{(2)}_{m\times n}$, $B^{(2)}_{1\times n}$와 활성화 함수 h에 의하여

$$\boxed{A = XW^{(1)} + B^{(1)}, \qquad Z = h(A), \qquad U = ZW^{(2)} + B^{(2)}, \qquad Y = \mathrm{softmax}(U)}$$

로 정의된다. 은닉층의 node 수는 m개이고, 출력층 U는 분류하고자 하는 class의 개수와 같은 n개의 node를 가진다. 정답을 나타내는 one-hot vector, softmax, 손실함수 l은 5–8절에서 설명한다.

변수 / parameter	shape	의미	MNIST 예
X	$1 \times l$	입력 (한 데이터)	1×784
$W^{(1)}$, $B^{(1)}$	$l \times m$, $1 \times m$	첫 번째 affine 계층	784×100, 1×100
A, Z	$1 \times m$	은닉층 (활성화 전/후)	1×100
$W^{(2)}$, $B^{(2)}$	$m \times n$, $1 \times n$	두 번째 affine 계층	100×10, 1×10
U	$1 \times n$	각 class의 점수 (score)	1×10
Y	$1 \times n$	각 class일 확률	1×10

지난주와 마찬가지로 데이터 하나를 **행벡터**로 쓰고 XW의 순서로 곱한다. 이 규약에서 가중치 행렬의 shape은 항상

$$\boxed{W : (\text{입력 node 수}) \times (\text{출력 node 수})} \qquad \big(W^{(1)} : l \times m, \quad W^{(2)} : m \times n\big)$$

이다. NumPy에서도 `W1.shape`은 `(784, 100)`, `W2.shape`은 `(100, 10)`이다. 그러면 11절에서 데이터(example) h개를 행으로 쌓은 행렬 $X_{h \times l}$에 대해서도

$$\underbrace{X}_{\text{(example 수)}\times\text{(입력 node 수)}} \quad \underbrace{W}_{\text{(입력 node 수)}\times\text{(출력 node 수)}} = \underbrace{XW}_{\text{(example 수)}\times\text{(출력 node 수)}}$$

이 되어, 같은 식 $XW + B$를 그대로 쓸 수 있다. 결과의 각 **행**은 각 데이터, 각 **열**은 다음 층의 각 node에 대응한다.

이 network는 몇 층인가? — 깊이와 넓이

A와 Z는 같은 은닉층의 활성화 **전**과 **후**의 값이므로 하나의 층으로 센다. U는 class별 점수를 내는 **출력층**이다.

- **깊이(depth)**는 층(layer)의 개수이다. 보통 입력층은 세지 않고, **가중치를 가진 층**(affine 계층)의 개수를 센다. 위 network는 은닉층 1개와 출력층 1개, 즉 가중치 행렬 $W^{(1)}, W^{(2)}$를 가진 두 층이므로 **2층 신경망(two-layer network)**이다. (입력층까지 세어 "3층"이라 부르는 책도 있으므로, 층을 세는 기준을 확인해야 한다.)
- **넓이(width)**는 한 층의 node 수이다. 위 network의 은닉층 넓이는 m (MNIST 예에서는 100)이다.
- 은닉층이 두 개 이상인, 즉 층이 깊은 network를 **심층신경망(deep neural network)**이라 한다.

NumPy 배열의 "차원"과 벡터의 "차원"

NumPy에서 말하는 배열의 차원과 수학에서 말하는 벡터의 차원은 다르다.

- `X = np.array([x1, x2, ..., xl])`는 **1차원 배열**이고 `X.shape`는 `(l,)`이다. 수학에서는 원소가 l개이므로 l차원 벡터라 부른다.
- `X = np.array([[x1, x2, ..., xl]])`는 1차원 배열 하나를 원소로 갖는 **2차원 배열**이고 `X.shape`는 `(1, l)`이다.

배열의 차원은 `X.ndim`, 모양은 `X.shape`로 확인한다.

```
import numpy as np
x = np.array([[1, 2, 3], [4, 5, 6]])
print(x.ndim, x.shape)        # 2 (2, 3)
y = np.array([[[1], [2], [3]], [[4], [5], [6]]])
print(y.ndim, y.shape)        # 3 (2, 3, 1)
```

우리는 데이터 하나를 `(1, l)`, 데이터 h개를 `(h, l)` 모양의 2차원 배열로 다룬다.

2 Affine 계층의 역전파 (입력층과 은닉층 사이)

2.1 성분으로 쓴 affine 변환

신경망의 순전파에서 먼저 선형 연결층을 표현해 보자. 편의상 층 번호를 나타내는 위첨자 (1)은 생략한다. $A = XW + B$에서

$$X_{1\times l} = (x_1, \ldots, x_l), \qquad A_{1\times m} = (a_1, \ldots, a_m), \qquad B_{1\times m} = (b_1, \ldots, b_m),$$

$$W_{l\times m} = (w_{ij}) = \begin{pmatrix} w_{11} & w_{12} & \cdots & w_{1m} \\ w_{21} & w_{22} & \cdots & w_{2m} \\ \vdots & & & \vdots \\ w_{l1} & w_{l2} & \cdots & w_{lm} \end{pmatrix}$$

이다. $A = XW + B$와 같은 일차 변환을 **affine 변환**이라 하며, 이는 1변수 일차함수 $a = xw + b$를 다변수 벡터함수로 일반화한 것이다. 행렬곱을 전개하여 성분끼리 비교하면

$$\begin{cases} a_1 = x_1w_{11} + x_2w_{21} + \cdots + x_lw_{l1} + b_1 \\ a_2 = x_1w_{12} + x_2w_{22} + \cdots + x_lw_{l2} + b_2 \\ \quad\vdots \\ a_m = x_1w_{1m} + x_2w_{2m} + \cdots + x_lw_{lm} + b_m \end{cases} \quad \text{즉} \quad \boxed{a_j = \sum_{i=1}^{l} x_iw_{ij} + b_j} \tag{1}$$

이다 $(i = 1, \ldots, l,\ j = 1, \ldots, m)$. 가중치 w_{ij}는 입력 node x_i와 은닉 node a_j를 연결하고, b_j는 a_j에만 연결된다.

2.2 무엇을 구하려는가

뒤에서 정의할 손실을 l이라 하자. A층까지 어떤 복잡한 계산을 거쳐 역전파된 기울기

$$\frac{\partial l}{\partial A} = \left(\frac{\partial l}{\partial a_1}, \frac{\partial l}{\partial a_2}, \dots, \frac{\partial l}{\partial a_m}\right)_{1\times m}$$

를 이미 알고 있다고 하자 (뒤쪽 층에서 전달받은 **upstream gradient**). 목표는

$$\frac{\partial l}{\partial X} = \left(\frac{\partial l}{\partial x_1}, \dots, \frac{\partial l}{\partial x_l}\right)_{1\times l}, \qquad \frac{\partial l}{\partial W} = \left(\frac{\partial l}{\partial w_{ij}}\right)_{l\times m}, \qquad \frac{\partial l}{\partial B} = \left(\frac{\partial l}{\partial b_1}, \dots, \frac{\partial l}{\partial b_m}\right)_{1\times m}$$

를 $\frac{\partial l}{\partial A}$로 나타내는 것이다. 각 기울기는 해당 변수와 **같은 shape**을 가진다는 점에 주목하자.

2.3 $\partial l/\partial X$: 모든 경로의 기여를 더한다

식 (1)에서 x_i는 은닉층의 **모든** node $a_1, \dots, a_m$을 거쳐 l과 연결되고 $\frac{\partial a_j}{\partial x_i} = w_{ij}$이므로, 합성함수 미분법에 의해

$$\frac{\partial l}{\partial x_i} = \sum_{j=1}^{m} \frac{\partial a_j}{\partial x_i}\frac{\partial l}{\partial a_j} = w_{i1}\frac{\partial l}{\partial a_1} + w_{i2}\frac{\partial l}{\partial a_2} + \cdots + w_{im}\frac{\partial l}{\partial a_m} = \left(\frac{\partial l}{\partial a_1}, \dots, \frac{\partial l}{\partial a_m}\right)\begin{pmatrix} w_{i1} \\ \vdots \\ w_{im} \end{pmatrix}$$

이다. 이를 $i = 1, \dots, l$에 대해 한꺼번에 나타내면

$$\frac{\partial l}{\partial X} = \left(\frac{\partial l}{\partial a_1}, \dots, \frac{\partial l}{\partial a_m}\right)\begin{pmatrix} w_{11} & w_{21} & \cdots & w_{l1} \\ \vdots & & & \vdots \\ w_{1m} & w_{2m} & \cdots & w_{lm} \end{pmatrix} = \frac{\partial l}{\partial A}\, W^{\mathsf{T}}$$

이다. shape으로 쓰면 $\left(\frac{\partial l}{\partial X}\right)_{1\times l} = \left(\frac{\partial l}{\partial A}\right)_{1\times m}\left(W^{\mathsf{T}}\right)_{m\times l}$이다.

2.4 $\partial l/\partial W$: 입력과 upstream gradient의 곱

가중치 w_{ij}는 x_i와의 곱으로 a_j에만 연결되므로

$$\frac{\partial l}{\partial w_{ij}} = \frac{\partial a_j}{\partial w_{ij}}\frac{\partial l}{\partial a_j} = x_i\frac{\partial l}{\partial a_j}$$

이다. 행렬로 모두 나타내면

$$\frac{\partial l}{\partial W} = \begin{pmatrix} x_1\frac{\partial l}{\partial a_1} & x_1\frac{\partial l}{\partial a_2} & \cdots & x_1\frac{\partial l}{\partial a_m} \\ x_2\frac{\partial l}{\partial a_1} & x_2\frac{\partial l}{\partial a_2} & \cdots & x_2\frac{\partial l}{\partial a_m} \\ \vdots & & & \vdots \\ x_l\frac{\partial l}{\partial a_1} & x_l\frac{\partial l}{\partial a_2} & \cdots & x_l\frac{\partial l}{\partial a_m} \end{pmatrix}_{l\times m} = \begin{pmatrix} x_1 \\ \vdots \\ x_l \end{pmatrix}_{l\times 1}\left(\frac{\partial l}{\partial a_1}, \dots, \frac{\partial l}{\partial a_m}\right)_{1\times m} = X^{\mathsf{T}}\frac{\partial l}{\partial A}$$

이다. ($l \times 1$ 행렬과 $1 \times m$ 행렬의 곱이 $l \times m$ 행렬이 된다.)

2.5 $\partial l/\partial B$: 그대로 통과

b_j 는 a_j 에만 연결되고 $\frac{\partial a_j}{\partial b_j} = 1$ 이므로 $\frac{\partial l}{\partial b_j} = \frac{\partial l}{\partial a_j}$, 즉

$$\frac{\partial l}{\partial B} = \frac{\partial l}{\partial A}$$

이다.

Affine 계층의 역전파 규칙

순전파: 각 행렬의 shape을 아래첨자로 쓰면

$$\boxed{A_{1\times m} = X_{1\times l} W_{l\times m} + B_{1\times m}}$$

역전파: 뒤에서 전달받은 $\frac{\partial l}{\partial A}$ 로부터

$$\boxed{\begin{aligned}\left(\frac{\partial l}{\partial X}\right)_{1\times l} &= \left(\frac{\partial l}{\partial A}\right)_{1\times m} (W^{\mathsf{T}})_{m\times l} \\ \left(\frac{\partial l}{\partial W}\right)_{l\times m} &= (X^{\mathsf{T}})_{l\times 1} \left(\frac{\partial l}{\partial A}\right)_{1\times m} \\ \left(\frac{\partial l}{\partial B}\right)_{1\times m} &= \left(\frac{\partial l}{\partial A}\right)_{1\times m}\end{aligned}}$$

를 계산한다. 각 기울기는 **해당 변수와 같은 shape**을 가진다 ($\frac{\partial l}{\partial X}$ 는 X 와, $\frac{\partial l}{\partial W}$ 는 W 와, $\frac{\partial l}{\partial B}$ 는 B 와 같은 shape). $\frac{\partial l}{\partial X}$ 는 앞쪽 층으로 다시 전달되고, $\frac{\partial l}{\partial W}$, $\frac{\partial l}{\partial B}$ 는 경사하강법에 사용된다.

역전파 식을 기억하는 법: 등식을 넘어가면 transpose

순전파 $A = XW$ 와 역전파 식을 나란히 놓아 보자.

$$A = XW \implies \frac{\partial l}{\partial X} = \frac{\partial l}{\partial A} W^{\mathsf{T}} \quad (W\text{가 넘어가면서 transpose})$$

$$A = XW \implies \frac{\partial l}{\partial W} = X^{\mathsf{T}} \frac{\partial l}{\partial A} \quad (X\text{가 넘어가면서 transpose})$$

순전파의 변수 A, X, W 는 역전파에서 **분모**로 들어가 $\frac{\partial l}{\partial A}, \frac{\partial l}{\partial X}, \frac{\partial l}{\partial W}$ 가 된다. 이때 구하려는 변수(X 또는 W)와 곱해진 행렬이 우변으로 등식을 타고 넘어가면서, $\frac{\partial l}{\partial A}$ 와 곱해진다고 기억하자.
왜 transpose가 되는지는 shape을 맞춰 보면 바로 알 수 있다. $\frac{\partial l}{\partial A}$ 는 $1 \times m$, W 는 $l \times m$ 이므로 $1 \times l$ 인 $\frac{\partial l}{\partial X}$ 를 만들 수 있는 곱은

$$\underbrace{\frac{\partial l}{\partial A}}_{1\times m} \underbrace{W^{\mathsf{T}}}_{m\times l} = \underbrace{\frac{\partial l}{\partial X}}_{1\times l}$$

뿐이고, X 는 $1 \times l$ 이므로 $l \times m$ 인 $\frac{\partial l}{\partial W}$ 를 만들 수 있는 곱은

$$\underbrace{X^{\mathsf{T}}}_{l\times 1} \underbrace{\frac{\partial l}{\partial A}}_{1\times m} = \underbrace{\frac{\partial l}{\partial W}}_{l\times m}$$

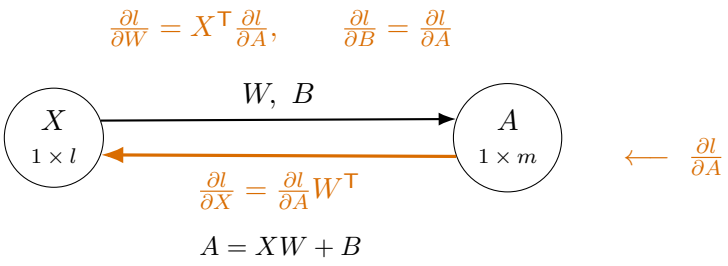
뿐이다. 즉 **shape만 맞추면 역전파 식은 저절로 결정된다.** (물론 2.3–2.5절처럼 성분으로 계산한 결과와 정확히 일치한다.)

2.6 계산 그래프를 그리는 두 가지 방법

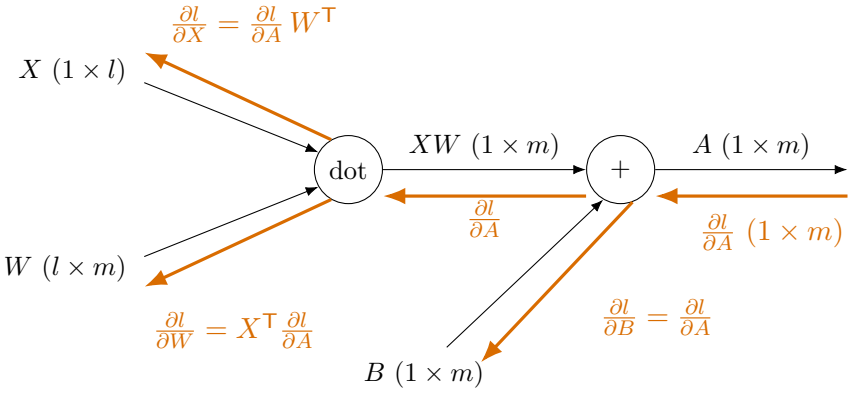
계산 그래프는 두 가지 방식으로 그릴 수 있다.

- **변수 노드 기준**: 원(노드) 안에 **변수**($X, A, Z, \ldots$)를 놓고, 노드 사이의 화살표 위에 그 변수들을 잇는 **연산과 parameter**(W, B)를 표시한다. 신경망을 그릴 때 neuron을 원으로 그리는 것과 같은 방식이며, 지금까지 우리 강의에서 사용해 온 방식이다.
- **연산 노드 기준**: 원(노드) 안에 **연산**(행렬곱 dot, 덧셈 $+, \ldots$)을 놓고, 화살표 위에 연산에 들어가고 나오는 **변수**를 표시한다. 입력 X 와 parameter W, B 가 모두 같은 자격의 "입력 화살표"로 그려지므로, W 와 B 로 흘러가는 역전파가 그림에 그대로 드러난다. PyTorch와 같은 프레임워크가 내부에서 자동미분(autograd)을 할 때 사용하는 방식이기도 하다.

두 그림은 같은 계산을 나타내며, 역전파 식도 똑같다. 검은 화살표는 순전파, 주황색 화살표는 역전파이다.



(1) 변수 노드 기준의 affine 계층



(2) 연산 노드 기준의 affine 계층 (괄호 안은 shape)

연산 노드 기준의 그림에서는 역전파를 **연산 노드마다 하나의 규칙**으로 읽을 수 있다. 각 노드는 출력 쪽(upstream)에서 흘러 들어온 gradient를 받아, 입력 쪽(downstream) 변수들로 전달한다.

- $+$ **노드** ($A = P + B$): $\frac{\partial A}{\partial P} = \frac{\partial A}{\partial B} = 1$ 이므로 upstream gradient를 **그대로** 두 입력에 전달한다.

$$\frac{\partial l}{\partial P} = \frac{\partial l}{\partial A}, \qquad \frac{\partial l}{\partial B} = \frac{\partial l}{\partial A}$$

- **dot 노드** ($P = XW$): upstream gradient $\frac{\partial l}{\partial P}$ 에 **반대편 입력의 transpose**를 행렬곱하여 전달한다.

$$\frac{\partial l}{\partial X} = \frac{\partial l}{\partial P} W^\top, \qquad \frac{\partial l}{\partial W} = X^\top \frac{\partial l}{\partial P}$$

곱하는 위치는 순전파 $P = XW$ 에서 상대 행렬이 있던 쪽이며, 앞의 "등식을 넘어가면 transpose"와 같은 규칙이다.

3 활성화 계층의 역전파

활성화 함수 h 에 의하여

$$Z = (z_1, \ldots, z_m) = h(A) = \big(h(a_1), \ldots, h(a_m)\big)$$

로 연결되어 있다고 하자. Z 층까지의 역전파 $\frac{\partial l}{\partial Z} = \left(\frac{\partial l}{\partial z_1}, \ldots, \frac{\partial l}{\partial z_m}\right)$ 를 알고 있을 때 $\frac{\partial l}{\partial A}$ 를 구하자. a_j 는 z_j 에**만** 연결되므로 합은 필요 없고

$$\frac{\partial l}{\partial a_j} = \frac{\partial z_j}{\partial a_j} \frac{\partial l}{\partial z_j} = h'(a_j) \frac{\partial l}{\partial z_j}$$

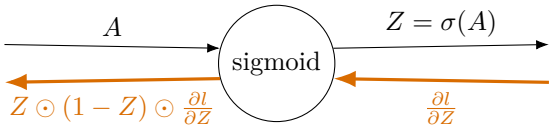
이다. 즉 활성화 계층의 역전파는 **성분별 곱**이다.

3.1 Sigmoid 활성화 함수

$h(a) = \sigma(a) = \dfrac{1}{1+e^{-a}}$ 이면 $\frac{\partial z}{\partial a} = z(1-z)$ 이므로

$$\frac{\partial l}{\partial A} = \left(z_1(1-z_1)\frac{\partial l}{\partial z_1}, \ldots, z_m(1-z_m)\frac{\partial l}{\partial z_m} \right) = \boxed{Z \odot (1-Z) \odot \frac{\partial l}{\partial Z}}$$

이다. 여기서 $\odot$는 성분끼리의 곱(아다마르 곱)이고, 1은 Z와 같은 shape의 모든 성분이 1인 행렬이다. NumPy에서는 `Z * (1 - Z) * dZ`처럼 `*`로 쓴다.



sigmoid 계층: 순전파의 출력 Z만 기억해 두면 역전파를 계산할 수 있다.

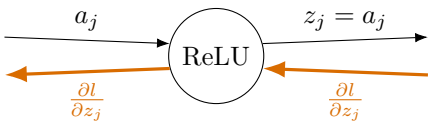
3.2 ReLU 활성화 함수

$$h(a) = \max(0,a) = \begin{cases} a, & a > 0 \\ 0, & a \le 0 \end{cases} \text{ 이면 } h'(a) = \begin{cases} 1, & a > 0 \\ 0, & a < 0 \end{cases} \text{ 이므로}$$

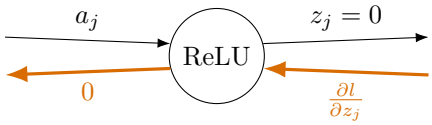
$$\frac{\partial l}{\partial a_j} = \begin{cases} \frac{\partial l}{\partial z_j}, & a_j > 0 \\ 0, & a_j \le 0 \end{cases} \quad \Longrightarrow \quad \boxed{\frac{\partial l}{\partial A} = \mathbf{1}[A > 0] \odot \frac{\partial l}{\partial Z}}$$

이다. 여기서 $\mathbf{1}[A > 0]$은 **지시함수(indicator function)**로, $a_j > 0$인 성분은 1, 나머지는 0인 A와 같은 shape의 행렬이다 (계단함수를 써서 $\text{step}(A)$로 쓰기도 한다). 꺾이는 점 $a_j = 0$에서는 $h'(0) = 0$으로 약속한다. 즉 ReLU는 입력이 양수였던 node의 기울기는 **그대로 통과**시키고, 입력이 0 이하였던 node의 기울기는 **0으로 막는다**. NumPy에서는 `np.where(A > 0, dZ, 0)`이다.

$a_j > 0$: 기울기를 그대로 통과



$a_j \le 0$: 기울기를 0으로 막음



ReLU 계층: 순전파 때 $a_j > 0$이었는지(`mask`)만 기억해 두면 된다.

이렇게 계산된 $\frac{\partial l}{\partial A}$는 2절의 affine 계층으로 전달되어 $\frac{\partial l}{\partial W^{(1)}}$, $\frac{\partial l}{\partial B^{(1)}}$, $\frac{\partial l}{\partial X}$를 계산하는 데 쓰인다.

4 Affine 계층의 역전파 (은닉층과 출력층 사이)

$Z_{1\times m}$ 에서 $U=(u_1,\dots,u_n)$ 로의 affine 변환

$$U_{1\times n} = Z_{1\times m}W^{(2)}_{m\times n} + B^{(2)}_{1\times n}$$

도 2 절과 **똑같은** 계층이다. 따라서 U 층까지의 역전파 $\frac{\partial l}{\partial U}$ 로부터

$$\frac{\partial l}{\partial Z} = \frac{\partial l}{\partial U}\,W^{(2)\mathsf{T}}, \qquad \frac{\partial l}{\partial W^{(2)}} = Z^{\mathsf{T}}\frac{\partial l}{\partial U}, \qquad \frac{\partial l}{\partial B^{(2)}} = \frac{\partial l}{\partial U}$$

를 계산할 수 있다. $\frac{\partial l}{\partial Z}$ 는 3 절의 활성화 계층으로 전달되고, $\frac{\partial l}{\partial W^{(2)}}$, $\frac{\partial l}{\partial B^{(2)}}$ 는 경사하강법에 사용된다. 같은 계층을 반복해서 쌓기 때문에 한 번 유도한 규칙을 몇 번이고 다시 쓸 수 있다. 이것이 역전파 알고리즘을 **계층(layer) 단위**로 구현하는 이유이다.

남은 것은 마지막 $U\to Y\to l$ 부분, 즉 $\frac{\partial l}{\partial U}$ 이다. 그 전에 먼저 출력 U 를 정답과 어떻게 비교할지부터 정해야 한다.

5 출력층의 설계: 정답을 어떻게 나타내고 무엇과 비교할까?

5.1 출력층 설계의 두 가지 질문

은닉층까지의 계산은 모두 정했다. 이제 network의 **마지막 층**을 설계할 차례이다. 학습이란 결국 "network의 출력이 정답에 가까워지도록" parameter를 바꾸는 일이므로, 출력층을 설계할 때는 다음 두 질문에 답해야 한다.

출력층 설계의 핵심 질문

Q1. network의 **출력**과 **정답**을 각각 어떤 형태로 나타내야 서로 비교할 수 있을까?

Q2. 출력과 정답의 차이를 어떤 **손실함수**로 잴까? (그리고 그 손실의 gradient는 학습하기에 좋은 모양일까?)

5.2 회귀에서는 두 질문이 쉬웠다

지난주의 회귀 문제를 떠올려 보자. network의 출력 $\hat{y}$는 실수 하나이고, 정답 t도 실수 하나였다. 둘은 처음부터 **같은 종류의 양**이므로 바로 뺄 수 있었고 (Q1), 그 차이를 제곱한 $l=(\hat{y}-t)^2$을 손실로 쓰면 되었다 (Q2). 출력층에서는 마지막 affine 계층의 값을 그대로 출력하면 되었다 (항등함수).

그렇다면 정답이 "숫자 $0\sim9$ 중 어느 것인가"라는 **class**일 때는 어떻게 해야 할까?

5.3 첫 번째 시도: class 번호를 출력 하나로 맞힌다면?

가장 쉬운 방법은 회귀처럼 출력 node를 하나만 두고, class 번호 $0,1,\dots,9$를 정답 값으로 삼아 제곱 오차로 학습하는 것이다. 실제로 지난주 실습에서는 class $0,1,2,3$을 이렇게 계단 모양 함수로 회귀하였다. 하지만 MNIST에서는 이 방법에 근본적인 문제가 있다.

- **class 번호에 없던 크기와 순서가 생긴다.** 출력 하나로 번호를 맞히면 "3과 4는 가깝고 3과 8은 멀다"고 가정하는 셈이다. 그러나 손글씨 숫자에서 3은 모양이 비슷한 8과 더 헷갈리기 쉽다. 번호는 이름표일 뿐, 크기의 의미가 없다.
- **애매한 입력에서 엉뚱한 답이 나온다.** 3인지 8인지 헷갈리는 이미지에 대해 network는 오차를 줄이려고 두 값의 중간인 5.5 근처를 출력하게 되고, 이를 반올림하면 전혀 관계없는 5나 6이 된다.
- **얼마나 확신하는지를 나타낼 수 없다.** 출력 5.5만으로는 "3일 가능성 반, 8일 가능성 반"이라는 정보를 표현할 수 없다.

지난주 실습이 잘 된 것은 class들이 x축 위에 **순서대로 놓이도록** 데이터를 만들었기 때문이다. 일반적인 분류 문제에서는 class 사이에 그런 순서가 없다.

5.4 두 번째 아이디어: class마다 출력 node를 하나씩

그래서 출력층에 **class의 개수만큼** node를 둔다. 출력 $U = (u_1, \ldots, u_n)$의 k번째 성분 u_k는 "입력이 k번째 class일 가능성"을 나타내는 어떠한 **점수(score)**이길 바란다. 이왕이면 출력층으로 보고 입력된 이미지가 각각의 class에 속하는지와 그 "확률"을 출력해주면 더 좋겠다. class들은 서로 대등한 별개의 node가 되므로 번호의 크기나 순서는 아무 의미가 없어지고, 여러 class에 동시에 점수를 줄 수 있어 애매함도 표현할 수 있다.

이제 Q1의 절반, 즉 **정답의 표현**을 정하자. 출력이 n개의 성분을 가진 벡터이므로, 정답도 n개의 성분을 가진 벡터로 나타내야 비교할 수 있다.

class가 n개인 다중분류 문제에서 k번째 class의 정답(label, target)을 k번째 원소만 1이고 나머지는 모두 0인 벡터로 나타내는 방법을 **one-hot encoding**이라 하고, 이러한 벡터를 **one-hot vector**라 한다.

class (target)	one-hot vector $t = (t_1, \ldots, t_n)$
1	$(1, 0, 0, \ldots, 0, 0)$
2	$(0, 1, 0, \ldots, 0, 0)$
3	$(0, 0, 1, \ldots, 0, 0)$
$\vdots$	$\vdots$
$n-1$	$(0, 0, 0, \ldots, 1, 0)$
n	$(0, 0, 0, \ldots, 0, 1)$

예를 들어 MNIST에서 정답이 숫자 5인 이미지의 target은 (class $0, 1, \ldots, 9$의 순서로)

$$t = (0, 0, 0, 0, 0, 1, 0, 0, 0, 0)$$

이다. 모든 class가 서로 같은 거리에 있으므로 번호의 크기와 순서가 사라진다.

5.5 One-hot vector는 확률분포이다. 그런데 U는?

one-hot vector t의 성분은 모두 0 이상 1 이하이고 합이 1이다. 즉 t는 "정답 class일 확률이 1, 나머지 class일 확률이 0"인 **확률분포**로 볼 수 있다.

그런데 network의 출력 U는 affine 계층의 결과이므로 각 성분이 음수일 수도, 1보다 클 수도 있고, 합도 1이 아니다. 예를 들어 정답이 첫 번째 class일 때

$$U = (2.0,\ 1.0,\ 0.1), \qquad t = (1,\ 0,\ 0)$$

은 서로 다른 종류의 양이어서 그대로 비교하기 어렵다. 정답은 확률분포인데 출력은 점수이니, Q1의 나머지 절반인 **출력의 표현**이 아직 정해지지 않은 것이다.

5.6 출력층 설계의 답

이 고민의 결론은 다음과 같다.

출력층 설계의 답: one-hot / softmax / cross entropy

A1. **정답**은 one-hot vector t로 나타낸다 (확률분포). **출력**은 점수 U를 **softmax 계층**으로 확률분포 Y로 바꾼다 (6절). 이제 Y와 t는 같은 종류의 대상이다.

A2. 두 확률분포 Y와 t의 차이는 **cross entropy**로 잰다 (7절). 게다가 이 선택은 역전파를 $\frac{\partial l}{\partial U} = Y - t$라는 아주 간단한 식으로 만들어 준다 (8, 9절).

$$U\ (\text{점수}) \xrightarrow{\text{softmax}} Y\ (\text{확률분포}) \xrightarrow{\text{cross entropy}} l \longleftarrow t\ (\text{one-hot, 확률분포})$$

회귀와 비교하면, 문제의 종류에 따라 출력층의 세 가지 선택이 함께 정해진다는 것을 알 수 있다.

문제	출력의 표현	정답의 표현	손실함수
회귀	실수 (항등함수)	실수	제곱 오차
다중분류	확률분포 (softmax)	확률분포 (one-hot)	cross entropy

다음 절부터 softmax와 cross entropy를 차례로 살펴본다.

6 Softmax 계층: 점수를 확률분포로

6.1 정의와 성질

점수 $U = (u_1, \ldots, u_n)$를 확률분포로 바꾸기 위해 다음 함수를 사용한다. 변수 $Y = (y_1, \ldots, y_n)$가

$$y_i = \frac{e^{u_i}}{\sum_{k=1}^{n} e^{u_k}} = \frac{e^{u_i}}{e^{u_1} + e^{u_2} + \cdots + e^{u_n}}, \qquad i = 1, \ldots, n$$

로 $U = (u_1, \ldots, u_n)$의 함수로 정의하자. 그러면 Y는 다음 세 가지 성질을 만족한다.

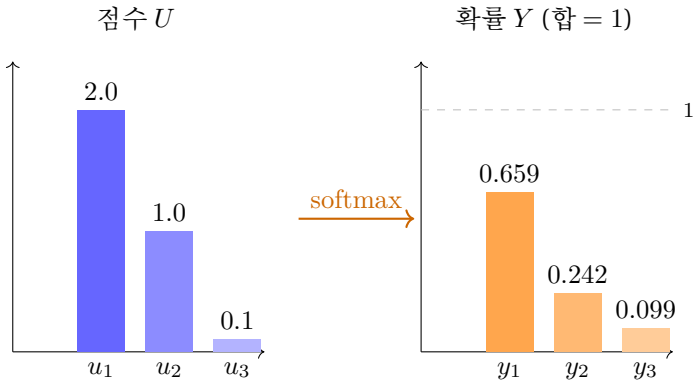
1) u_i의 크기 순서와 y_i의 크기 순서가 일치한다. 즉 $u_i < u_j$이면 $y_i < y_j$이다.

2) $0 < y_i < 1$이다.

3) $\sum_{i=1}^{n} y_i = 1$이다.

이렇게 정의된 U에서 Y로의 함수를 **softmax 함수**라 하고 $Y = \text{softmax}(U)$로 표시한다.

성질 2), 3) 때문에 y_k를 입력 X가 k **번째 class에 속할 확률**로 해석할 수 있다. 즉 출력층 U의 각 원소가 X가 k번째 class에 속할 "점수"라면, softmax는 이 점수를 "확률"로 바꾸어 주는 함수이다. 성질 1) 때문에 가장 큰 점수를 받은 class가 가장 큰 확률을 받는다. 이제 network의 출력 Y와 정답 t는 **둘 다 확률분포**이므로 서로 직접 비교할 수 있다. (두 확률 분포를 어떻게 비교할 수 있을까?)

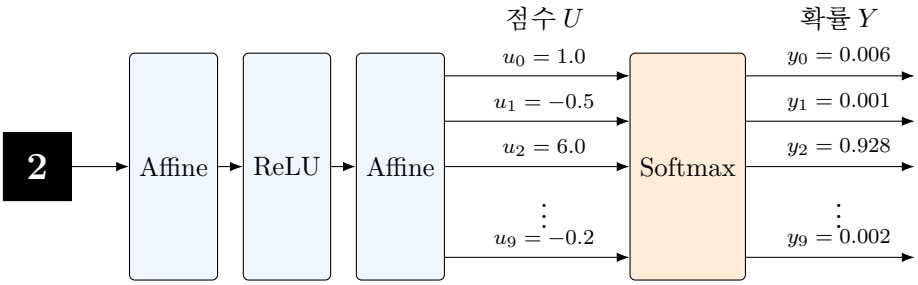


예를 들어 $U = (2.0,\ 1.0,\ 0.1)$이면 $e^2 \approx 7.389$, $e^1 \approx 2.718$, $e^{0.1} \approx 1.105$이므로

$$Y = \text{softmax}(U) \approx \left(\frac{7.389}{11.213}, \frac{2.718}{11.213}, \frac{1.105}{11.213}\right) \approx (0.659,\ 0.242,\ 0.099)$$

이다.

MNIST에서는 입력 이미지가 affine 계층과 ReLU 계층을 거쳐 10개 class의 점수 U가 되고, softmax 계층이 이 점수를 합이 1인 확률 Y로 바꾼다. 아래 예에서 숫자 '2'의 점수 6.0은 확률 0.928 (92.8%)로, 숫자 '0'의 점수 1.0은 확률 0.006으로 바뀐다.



학습이 끝난 뒤 **예측(추론)**만 할 때는 가장 큰 점수의 class만 고르면 되므로 softmax를 생략해도 된다 (성질 1)). softmax는 **학습**할 때 손실을 계산하기 위해 필요하다.

6.2 수치적으로 안전한 계산

e^u는 u가 조금만 커져도 매우 커져서 컴퓨터에서 overflow가 일어난다 (예: e^{1000}). 그런데 임의의 상수 c에 대하여

$$\frac{e^{u_i - c}}{\sum_k e^{u_k - c}} = \frac{e^{-c} e^{u_i}}{e^{-c} \sum_k e^{u_k}} = \frac{e^{u_i}}{\sum_k e^{u_k}}$$

이므로 $\text{softmax}(U - c) = \text{softmax}(U)$이다. 따라서 $c = \max_k u_k$를 빼고 계산하면 지수의 최댓값이 $e^0 = 1$이 되어 안전하다.

```
def softmax(U):
    U = U - np.max(U, axis=1, keepdims=True)    # same result, avoids overflow of exp
    expU = np.exp(U)
    return expU / np.sum(expU, axis=1, keepdims=True)
```

(`axis=1`은 한 데이터, 즉 한 **행** 안에서 n개 class에 대해 계산한다는 뜻이다.)

7 Cross entropy 손실함수

이제 softmax 출력 $Y = (y_1, \ldots, y_n)$와 one-hot vector $t = (t_1, \ldots, t_n)$는 둘 다 확률분포이다. 이 두 확률분포의 차이로 손실함수를 정의하자. 제곱 손실

$$l = \sum_{i=1}^{n} (y_i - t_i)^2$$

을 사용할 수도 있으나, 다중분류 문제에서는 **cross entropy error**를 사용하는 것이 일반적이다:

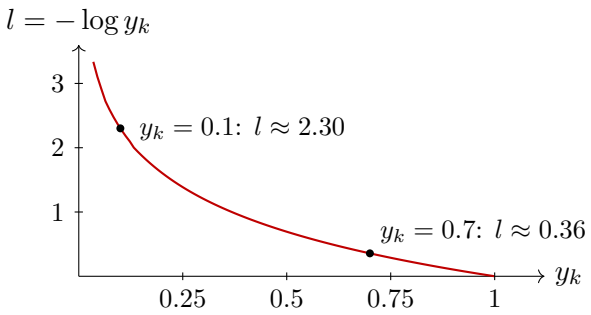
$$\boxed{l = -\sum_{i=1}^{n} t_i \log y_i}$$

여기서 log는 자연로그 ln이다. 지금까지는 제곱 오차를 써 왔는데 왜 갑자기 cross entropy를 쓰는지는 먼저 그 의미와 역전파를 살펴본 뒤 9절에서 자세히 이야기한다.

t가 k번째 class를 나타내는 one-hot vector라면 t_k만 1이고 나머지는 모두 0이므로 실제로는 하나의 항만 남아

$$l = -\log y_k$$

이다. 즉 cross entropy는 **정답 class에 준 확률의 로그에 마이너스를 붙인 값**이다.



$0 < y_k < 1$이므로 $l = -\log y_k > 0$이고, $y_k \to 1$일 때 $l \to 0$, $y_k \to 0$일 때 $l \to \infty$이다. 따라서 cross entropy 손실 l을 줄이도록 network를 훈련시킨다는 것은

- 정답 class의 node y_k의 값을 1에 가깝게 출력하도록 하고,
- Y의 원소의 합이 1이므로 나머지 node는 0에 가깝게 출력하도록,

즉 Y가 one-hot vector t에 가까워지도록 훈련하는 것이다. 그러면 훈련된 network의 출력 Y를 보고 입력 X가 어떤 class에 속할 확률이 가장 큰지 판단할 수 있다.

예를 들어 정답이 $t = (0, 1, 0)$ 일 때

$$Y = (0.1,\ 0.7,\ 0.2) \ \Rightarrow\ l = -\log 0.7 \approx 0.36, \qquad Y = (0.6,\ 0.3,\ 0.1) \ \Rightarrow\ l = -\log 0.3 \approx 1.20$$

이다. 정답에 낮은 확률을 준 두 번째 경우를 훨씬 크게 벌한다.

$\log 0$ 문제

y_k 가 0에 매우 가까우면 $\log y_k \to -\infty$ 가 되어 계산이 깨진다. 코드에서는 아주 작은 수를 더해 `np.log(Y + 1e-12)` 로 계산한다.

8 Softmax–cross entropy 계층의 역전파

이제 손실 l 의 u_i 에 대한 미분 $\frac{\partial l}{\partial u_i}$ 를 계산하자. u_i 는 softmax 때문에 Y 층의 **모든** node $y_1, \ldots, y_n$ 을 거쳐 l 에 연결되므로

$$\frac{\partial l}{\partial u_i} = \sum_{j=1}^{n} \frac{\partial y_j}{\partial u_i} \frac{\partial l}{\partial y_j}$$

이다.

8.1 Softmax의 미분 $\partial y_j / \partial u_i$

$S = e^{u_1} + \cdots + e^{u_n}$ 이라 하면 $y_j = e^{u_j}/S$ 이고 $\frac{\partial S}{\partial u_i} = e^{u_i}$ 이다. 몫의 미분법에 의해 두 가지 경우로 나뉜다.

(1) $j = i$ 인 경우

$$\frac{\partial y_i}{\partial u_i} = \frac{e^{u_i} S - e^{u_i} e^{u_i}}{S^2} = \frac{e^{u_i}}{S} - \left(\frac{e^{u_i}}{S}\right)^2 = y_i - y_i^2 = y_i(1 - y_i)$$

(2) $j \neq i$ 인 경우 (e^{u_j} 는 u_i 에 대해 상수)

$$\frac{\partial y_j}{\partial u_i} = \frac{0 \cdot S - e^{u_j} e^{u_i}}{S^2} = -\frac{e^{u_j}}{S} \cdot \frac{e^{u_i}}{S} = -y_j y_i$$

8.2 Cross entropy의 미분과 결합

$l = -\sum_j t_j \log y_j$ 에서 로그함수의 미분법에 의해

$$\frac{\partial l}{\partial y_j} = -\frac{t_j}{y_j}$$

이다. 따라서 합성함수 미분법에 의해

$$\begin{aligned}\frac{\partial l}{\partial u_i} &= \underbrace{y_i(1-y_i)\times\left(-\frac{t_i}{y_i}\right)}_{j=i} + \sum_{j\neq i}\underbrace{(-y_j y_i)\times\left(-\frac{t_j}{y_j}\right)}_{j\neq i} \\ &= (y_i-1)\,t_i + \sum_{j\neq i} y_i\,t_j = y_i\,(t_1+t_2+\cdots+t_n) - t_i \\ &= y_i - t_i\end{aligned}$$

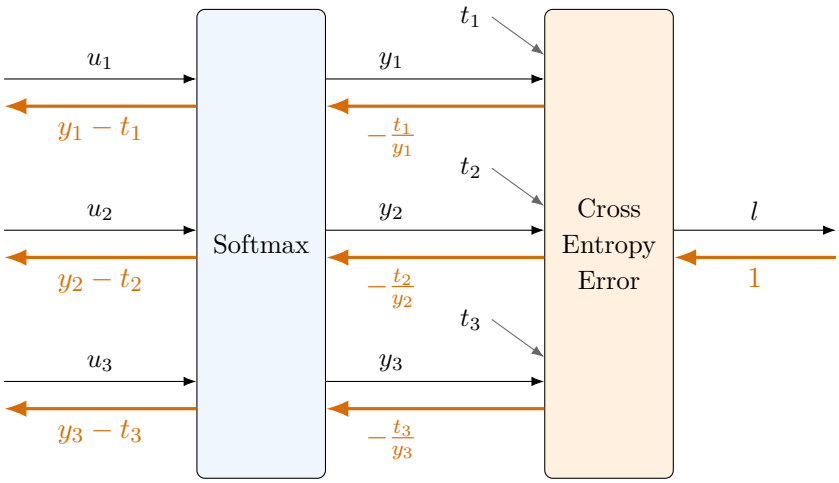
이다. 여기서 one-hot vector는 $t_1+t_2+\cdots+t_n=1$ 이라는 사실을 이용하였다. 따라서

Softmax–cross entropy 계층의 역전파

$$\frac{\partial l}{\partial U} = \left(\frac{\partial l}{\partial u_1},\ldots,\frac{\partial l}{\partial u_n}\right) = (y_1-t_1,\ldots,y_n-t_n) = Y-t$$

softmax 계층과 cross entropy 손실을 하나로 묶으면, 역전파는 **예측 확률에서 정답을 뺀 값**이라는 매우 간단한 형태가 된다.

여기서 손실 l의 softmax 층에 대한 미분 $\frac{\partial l}{\partial Y}$를 따로 계산해 전달하지 않고, softmax 층과 cross entropy 오차를 묶어서 "softmax–cross entropy" 층에 대한 역전파 $\frac{\partial l}{\partial U}$를 바로 나타냈음에 주목하자. $\frac{\partial l}{\partial Y} = -t/Y$와 $n\times n$ Jacobian $\frac{\partial Y}{\partial U}$를 각각 계산해 곱하는 것보다 훨씬 간단하고 수치적으로도 안정적이다. class가 3개인 경우의 계산 그래프로 나타내면 다음과 같다.



Softmax-with-Loss 계층: 역전파는 출력의 끝 $\frac{\partial l}{\partial l}=1$에서 출발하여 $Y-t$가 된다.

중간의 $-\frac{t_i}{y_i}$와 softmax의 Jacobian이 곱해지면서 복잡한 항들이 모두 소거되고, softmax 계층의 출력과 정답의 **차이** $y_i - t_i$만 앞 계층으로 전달된다. 예를 들어 정답이 $t=(0,1,0)$일 때

- network가 $Y=(0.3,\ 0.2,\ 0.5)$를 출력하면 (제대로 인식하지 못함) $\frac{\partial l}{\partial U}=(0.3,\ -0.8,\ 0.5)$라는 **큰** 기울기가 전달되어 많이 배우고,
- $Y=(0.01,\ 0.99,\ 0)$을 출력하면 (거의 맞힘) $\frac{\partial l}{\partial U}=(0.01,\ -0.01,\ 0)$이라는 **작은** 기울기가 전달되어 조금만 배운다.

$Y - t$의 의미

정답 class k에 대해서는 $\frac{\partial l}{\partial u_k} = y_k - 1 < 0$이므로 경사하강법은 점수 u_k를 **올리고**, 나머지 class에 대해서는 $\frac{\partial l}{\partial u_j} = y_j > 0$이므로 점수 u_j를 **내린다**. 그리고 그 크기는 "얼마나 틀렸는가"에 비례한다. 지난주 MSE에서의 $\frac{\partial l}{\partial y} = 2(y - t)$와 비슷한 모양이라는 점도 확인하자.

9 왜 cross entropy를 쓰는가?

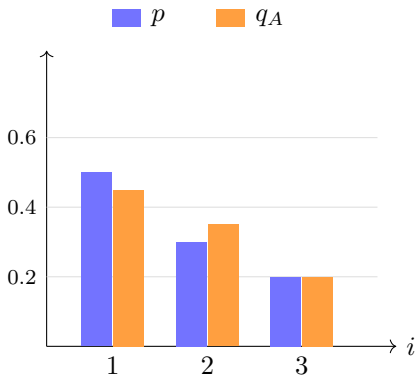
지난주까지는 제곱 오차를 손실함수로 사용했다. 그런데 다중분류에서는 왜 cross entropy를 쓸까? 세 가지 이유가 있다.

9.1 두 확률분포가 얼마나 다른지를 재는 척도이다

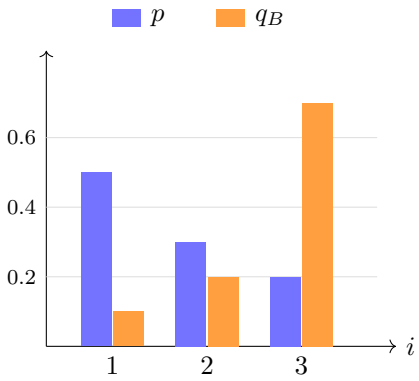
질문: 두 확률분포는 얼마나 닮았는가? 같은 표본공간 $\Omega = \{1, 2, 3\}$ 위의 이산확률분포 세 개를 생각하자. (두 분포를 비교하려면 **같은 표본공간** 위에서 정의되어 있어야 한다. 분류 문제에서는 표본공간이 class의 집합 $\{1, \ldots, n\}$이다.)

$$p = (0.5,\ 0.3,\ 0.2), \qquad q_A = (0.45,\ 0.35,\ 0.2), \qquad q_B = (0.1,\ 0.2,\ 0.7)$$

p를 기준이 되는 분포(정답 분포)라 하고, q_A, q_B를 각각 p와 겹쳐 확률질량함수를 그려 보면 다음과 같다.



(가) p와 q_A: 거의 같다



(나) p와 q_B: 많이 다르다

그림으로 보면 q_A는 p와 거의 같고 q_B는 p와 많이 다르다. 그렇다면 "두 분포가 얼마나 다른가"를 **하나의 수**로 나타내려면 어떻게 해야 할까? softmax가 내놓는 분포 Y를 정답 분포 t에 가깝게 만드는 것이 학습의 목표이므로, 이 질문에 대한 답이 곧 손실함수가 된다.

가장 먼저 떠오르는 방법은 성분별 차이의 제곱합 $\sum_i (p_i - q_i)^2$이다. 이것도 하나의 척도이지만, 확률분포라는 특성을 쓰지 않고 p, q를 그냥 벡터로 본 것이다. 확률분포에 걸맞은 척도는 "**놀라움(surprise)**"이라는 개념에서 출발한다.

놀라움과 entropy. 확률이 q_i인 사건이 일어났을 때의 놀라움을

$$-\log q_i$$

로 재자. 확률이 1인 사건이 일어나면 놀라움은 0이고, 확률이 작은 사건일수록 놀라움이 크다 ($q_i \to 0$이면 $-\log q_i \to \infty$). 결과가 실제로 분포 p를 따라 일어날 때, 평균 놀라움

$$H(p) = -\sum_{i=1}^{n} p_i \log p_i$$

를 p의 **entropy**라 한다. 분포 p 자체가 가진 불확실성의 크기이다.

Cross entropy: 틀린 분포로 놀라움을 잴 때. 그런데 결과는 실제로 p를 따라 일어나는데, 우리가 그 분포를 q라고 **믿고** 놀라움을 $-\log q_i$로 잰다면 평균 놀라움은

$$H(p, q) = -\sum_{i=1}^{n} p_i \log q_i$$

가 된다. 이것을 p에 대한 q의 **cross entropy**라 한다. q가 실제로 자주 일어나는 결과(p_i가 큰 i)에 작은 확률을 줄수록 큰 놀라움이 자주 생겨 $H(p, q)$가 커진다.

KL divergence: 틀린 믿음 때문에 추가된 놀라움. q가 p와 정확히 같으면 cross entropy는 entropy와 같아진다: $H(p, p) = H(p)$. 그 차이

$$\boxed{D_{\mathrm{KL}}(p \,\|\, q) = H(p, q) - H(p) = \sum_{i=1}^{n} p_i \log \frac{p_i}{q_i}}$$

를 **KL divergence**(Kullback–Leibler divergence)라 한다. p 대신 q를 믿었기 때문에 **추가로** 생긴 평균 놀라움이며,

$$D_{\mathrm{KL}}(p \,\|\, q) \geq 0, \qquad D_{\mathrm{KL}}(p \,\|\, q) = 0 \iff p = q$$

이다. 즉 KL divergence는 두 분포가 다를수록 커지는, 두 확률분포의 차이를 재는 척도이다.

$D_{\mathrm{KL}}(p \,\|\, q) \geq 0$의 증명

모든 $x > 0$에 대하여 $\log x \leq x - 1$이고 등호는 $x = 1$일 때만 성립한다. 따라서

$$-D_{\mathrm{KL}}(p \,\|\, q) = \sum_i p_i \log \frac{q_i}{p_i} \leq \sum_i p_i \left(\frac{q_i}{p_i} - 1 \right) = \sum_i q_i - \sum_i p_i = 1 - 1 = 0$$

이고, 등호는 모든 i에 대해 $q_i/p_i = 1$, 즉 $p = q$일 때만 성립한다. (합은 $p_i > 0$인 i에 대해서만 한다.) KL divergence는 $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$일 수 있으므로 엄밀한 의미의 거리는 아니지만, 두 분포의 차이를 재는 척도로 널리 쓰인다.

위의 예에서 실제로 계산해 보면 다음과 같다 ($H(p) \approx 1.030$).

	cross entropy $H(p, q)$	KL divergence $D_{\mathrm{KL}}(p\|q)$	(참고) $\sum_i (p_i - q_i)^2$
$q_A = (0.45, 0.35, 0.2)$	1.036	0.006	0.005
$q_B = (0.1, 0.2, 0.7)$	1.706	0.676	0.420

그림에서 본 직관대로 q_B가 p에서 훨씬 멀다. q_B는 p에서 가장 자주 일어나는 결과 1 ($p_1 = 0.5$)에 확률 0.1만 주었으므로, 그 결과가 일어날 때마다 놀라움이 $-\log 0.1 \approx 2.30$으로 크다.

정답 분포가 고정되어 있으면: KL을 줄이는 것 = cross entropy를 줄이는 것. 위 식을 다시 쓰면

$$H(p,q) = \underbrace{H(p)}_{p\text{만의 함수}} + D_{\mathrm{KL}}(p \,\|\, q)$$

이다. 학습에서 p는 **정답 분포** t로 **고정**되어 있고, 우리가 바꿀 수 있는 것은 network의 출력 분포 $q = Y = \mathrm{softmax}(U)$ 뿐이다. p가 고정되면 entropy $H(p)$는 **상수**이므로,

$$\text{KL divergence } D_{\mathrm{KL}}(t \,\|\, Y)\text{를 작게 하는 것} \iff \text{cross entropy } H(t,Y)\text{를 작게 하는 것}$$

이다. 두 분포의 차이(KL divergence)를 줄이는 방법은 cross entropy를 줄이는 것뿐이며, cross entropy가 가장 작아지는 것은 $Y = t$일 때이다. 그래서 cross entropy를 손실함수로 쓴다.

One-hot 정답이면. 정답이 k번째 class인 one-hot vector t이면 $t_k = 1$이고 나머지는 0이므로 $H(t) = -1 \cdot \log 1 = 0$이다 (단, $0 \log 0 = 0$으로 약속한다). 따라서

$$l = H(t,Y) = D_{\mathrm{KL}}(t \,\|\, Y) = -\log y_k \qquad (k: \text{ 정답 class})$$

가 되어 cross entropy와 KL divergence가 정확히 같아진다. 두 분포를 비교하는 일반적인 척도가, one-hot 정답과 만나면 **정답 node의 확률 하나만 보는** 7절의 간단한 해석이 되는 것이다.

최대가능도(maximum likelihood)의 관점

y_k는 network가 정답 class에 준 확률이므로, 데이터 N개에 대한 평균 손실 $\frac{1}{N}\sum_i \left(-\log y_{k_i}^{(i)}\right) = -\frac{1}{N}\log \prod_i y_{k_i}^{(i)}$을 최소화하는 것은 network가 **훈련 데이터의 정답들을 모두 맞힐 확률** $\prod_i y_{k_i}^{(i)}$을 최대화하는 것과 같다. 통계학에서 말하는 최대가능도 추정이다.

9.2 Softmax와 짝을 이루어 역전파가 간단해진다

8절에서 보았듯이 softmax와 cross entropy를 묶으면

$$\frac{\partial l}{\partial U} = Y - t$$

라는 매우 간단한 역전파를 얻는다. 이것은 우연이 아니다. 지난주의 회귀 문제에서 출력층이 항등함수 $y = u$이고 손실이 제곱 오차일 때에도 $\frac{\partial l}{\partial u} = 2(y - t)$로 "출력 − 정답" 꼴이었다.

문제	출력층	손실함수
회귀	항등함수	제곱 오차
다중분류	softmax	cross entropy

이처럼 출력층의 함수와 손실함수를 **짝**으로 고르면 역전파가 "출력과 정답의 차이"로 깔끔하게 떨어진다.

9.3 제곱 오차로도 학습할 수 있을까?

물론 가능하다. softmax 출력에 제곱 오차 $l = \frac{1}{2}\sum_i (y_i - t_i)^2$ 를 쓰더라도 합성함수 미분법으로 역전파를 계산할 수 있다. $\frac{\partial l}{\partial y_j} = y_j - t_j$ 와 softmax의 미분($\frac{\partial y_j}{\partial u_i}$, 8절)을 결합하면

$$\frac{\partial l}{\partial u_i} = \sum_{j=1}^{n} \frac{\partial y_j}{\partial u_i}(y_j - t_j) = y_i\Big((y_i - t_i) - \sum_{j=1}^{n} y_j(y_j - t_j)\Big)$$

이다 (연습문제 8). 실습 코드의 `SoftmaxWithSquaredError` 계층이 이 식을 구현한 것이며, 기울기 검사도 통과한다. 그러나 cross entropy와 비교하면 다음과 같은 약점이 있다.

- **확신을 갖고 틀리면 거의 배우지 못한다.** 위 식에는 y_i 가 곱해져 있어서, 정답 class의 확률 y_k 가 0에 가까우면 기울기도 0에 가까워진다. 예를 들어 $t = (1, 0)$ 인데 network가 $Y = (0.01,\ 0.99)$ 를 출력하면

  $$\text{cross entropy: } \frac{\partial l}{\partial U} = (-0.99,\ 0.99), \qquad \text{제곱 오차: } \frac{\partial l}{\partial U} \approx (-0.0196,\ 0.0196)$$

  로, 가장 많이 배워야 할 순간에 제곱 오차의 기울기는 **약 50배 작다.**

- **학습 초기가 느리다.** 초기에는 모든 class의 확률이 $y_i \approx \frac{1}{n}$ 로 비슷하므로, y_i 가 곱해진 제곱 오차의 기울기는 작다.

실제로 scikit-learn에 들어 있는 8×8 손글씨 숫자 데이터(훈련 1500개, test 297개)로 같은 network(은닉 node 50개, 미니배치 50, 학습률 0.5)를 학습시키면 test 정확도는 다음과 같다.

손실함수	5 epoch	10 epoch	20 epoch	30 epoch
softmax + cross entropy	91.6%	96.3%	97.3%	97.6%
softmax + 제곱 오차	22.6%	25.6%	91.6%	94.6%

제곱 오차도 결국 학습은 되지만 처음 10 epoch 동안 거의 진전이 없고, 최종 정확도도 낮다.

다중분류에서 cross entropy를 쓰는 이유

1. 정답 분포 t 와 출력 분포 Y 라는 **두 확률분포의 차이를 재는 척도**이며, one-hot 정답이면 $-\log y_k$ 로 간단해진다.
2. softmax와 짝을 이루어 역전파가 $\frac{\partial l}{\partial U} = Y - t$ 로 **간단명료**해진다.
3. 틀린 정도에 비례하는 기울기를 주므로 제곱 오차보다 **빠르고 안정적으로** 학습한다.

10 정리: 데이터 하나에 대한 순전파와 역전파

가중치와 편이를 초기화한 상태에서 하나의 데이터 X 와 정답 t 에 대하여

순전파와 역전파 한눈에 보기

순전파

$$A = XW^{(1)} + B^{(1)},\ Z = h(A),\ U = ZW^{(2)} + B^{(2)},\ Y = \mathrm{softmax}(U),\ l = -\sum_{i=1}^{n} t_i \log y_i$$

역전파 (순서를 거꾸로)

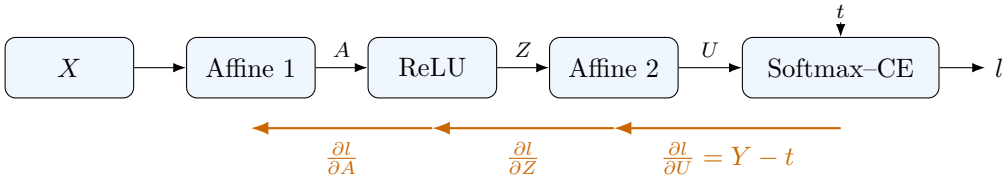
$$\frac{\partial l}{\partial U} = Y - t$$

$$\frac{\partial l}{\partial W^{(2)}} = Z^\top \frac{\partial l}{\partial U}, \qquad \frac{\partial l}{\partial B^{(2)}} = \frac{\partial l}{\partial U}, \qquad \frac{\partial l}{\partial Z} = \frac{\partial l}{\partial U} W^{(2)\top}$$

$$\frac{\partial l}{\partial A} = Z \odot (1 - Z) \odot \frac{\partial l}{\partial Z}\ \text{(sigmoid)} \quad \text{또는} \quad \frac{\partial l}{\partial A} = \mathbf{1}[A > 0] \odot \frac{\partial l}{\partial Z}\ \text{(ReLU)}$$

$$\frac{\partial l}{\partial W^{(1)}} = X^\top \frac{\partial l}{\partial A}, \qquad \frac{\partial l}{\partial B^{(1)}} = \frac{\partial l}{\partial A}, \qquad \frac{\partial l}{\partial X} = \frac{\partial l}{\partial A} W^{(1)\top}$$

이전 층까지의 역전파 결과가 다음 층의 역전파 계산에 그대로 쓰인다는 점에 주목하자. 이제 경사하강법으로 가중치와 편이를 갱신하면 된다.



11 미니배치 학습

11.1 온라인 학습, full-batch 학습, 미니배치 학습

지금까지 유도한 역전파는 **데이터 하나**에 대한 것이었다. 그런데 훈련 데이터는 N 개(MNIST에서는 60000 개)이다. 가중치를 한 번 업데이트할 때 **몇 개의 데이터**를 보고 gradient를 계산할지에 따라 학습 방법을 세 가지로 나눌 수 있다. 한 번에 사용하는 데이터의 개수를 h라 하고, 그 h 개에 대한 손실의 평균을

$$L = \frac{1}{h}\sum_{k=1}^{h} l_k, \qquad \frac{\partial L}{\partial W} = \frac{1}{h}\sum_{k=1}^{h} \frac{\partial l_k}{\partial W}, \qquad \frac{\partial L}{\partial B} = \frac{1}{h}\sum_{k=1}^{h} \frac{\partial l_k}{\partial B}$$

라 하면, 업데이트는 모두 $W \leftarrow W - \alpha\frac{\partial L}{\partial W}$, $B \leftarrow B - \alpha\frac{\partial L}{\partial B}$ 의 꼴이다.

1. **온라인 학습** (stochastic gradient descent, $h = 1$): 데이터를 **하나씩** 보고 그때마다 가중치를 업데이트한다. 과제 1처럼 손으로 계산할 때의 방식이다.
2. **Full-batch 학습** ($h = N$): **전체** 데이터의 평균 gradient를 계산한 뒤 가중치를 한 번 업데이트한다. 지난주 실습에서 데이터 12개를 한꺼번에 흘려보낸 방식이다.

3. **미니배치 학습** ($1 < h < N$): 전체 데이터에서 무작위로 고른 **작은 묶음**(미니배치) h 개의 평균 gradient로 업데이트한다. 보통 $h = 32 \sim 256$ 정도를 사용한다.

방법	장점	단점
온라인 ($h = 1$)	데이터 하나만 보면 바로 업데이트 할 수 있어 업데이트가 자주 일어나고, 메모리가 적게 든다.	gradient가 데이터 하나에 크게 좌우되어 **들쭉날쭉**하다. 데이터를 하나씩 처리하므로 행렬 연산의 병렬성을 살리지 못해 **느리다**.
Full-batch ($h = N$)	전체 데이터의 정확한 평균 gradient를 쓰므로 업데이트 방향이 **안정적**이다.	업데이트 **한 번**에 N 개를 모두 계산해야 한다. N 이 크면 한 걸음이 너무 느리고, 전체 데이터가 메모리에 들어가지 않을 수도 있다.
미니배치 ($1 < h < N$)	전체 평균의 **적당히 좋은 근사**를 **싸게** 얻는다. h 개를 행렬 한 번으로 계산하므로 빠르고, 1 epoch에 여러 번 (N/h 번) 업데이트한다.	gradient에 약간의 무작위성(잡음)이 남는다. h 라는 hyper-parameter를 정해야 한다.

왜 미니배치 학습을 주로 쓰는가?

- 전체 데이터의 평균 gradient는 일부 데이터의 평균으로도 꽤 잘 근사된다. 100개로 계산한 방향과 60000개로 계산한 방향이 크게 다르지 않다면, 60000개를 다 계산해서 한 걸음 가는 것보다 100개씩 보고 600걸음 가는 편이 훨씬 빨리 내려간다.
- 데이터 h 개를 **행렬 한 번**으로 처리하므로, 데이터를 하나씩 처리하는 온라인 학습보다 같은 양의 데이터를 훨씬 빠르게 계산한다 (다음 절).
- gradient에 남는 약간의 잡음은 오히려 학습에 도움이 되기도 한다.

즉 미니배치 학습은 full-batch의 **안정성**과 온라인 학습의 **잦은 업데이트** 사이의 절충이며, 현대 딥러닝은 거의 모두 미니배치 학습을 사용한다.

11.2 행렬 연산으로 여러 데이터를 한꺼번에 순전파

h 개의 데이터 $X_1, \ldots, X_h$ (각각 $1 \times l$)와 그 출력 $Y_k = X_k W + B$ (각각 $1 \times m$)를 행으로 쌓아

$$X = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1l} \\ x_{21} & x_{22} & \cdots & x_{2l} \\ \vdots & & & \vdots \\ x_{h1} & x_{h2} & \cdots & x_{hl} \end{bmatrix}_{h \times l} \quad \text{(미니배치 데이터)},$$

$$Y = \begin{bmatrix} y_{11} & \cdots & y_{1m} \\ \vdots & & \vdots \\ y_{h1} & \cdots & y_{hm} \end{bmatrix}_{h \times m} \quad \text{(미니배치의 출력)}$$

이라 하면, 행렬 연산으로

$$Y_{h\times m} = X_{h\times l}\, W_{l\times m} + B_{h\times m}, \qquad B_{h\times m} = \begin{bmatrix} b_1 & b_2 & \cdots & b_m \\ \vdots & & & \vdots \\ b_1 & b_2 & \cdots & b_m \end{bmatrix}$$

으로 여러 데이터를 한꺼번에 순전파할 수 있다. B는 같은 행 $(b_1, \ldots, b_m)$을 h번 복사한 것이며, NumPy에서는 $1 \times m$ 배열 `B`를 더하면 **broadcasting**으로 자동으로 행 방향으로 복사된다.

11.3 왜 행렬 연산으로 한꺼번에 계산하는가? — 병렬 연산과 GPU

$Y = XW + B$를 데이터 하나씩, 성분 하나씩 계산하면

$$y_{kj} = \sum_{i=1}^{l} x_{ki} w_{ij} + b_j \qquad (k = 1, \ldots, h,\ \ j = 1, \ldots, m)$$

을 $h \times m$번 반복해야 한다. 그런데 이 $h \times m$개의 값은 **서로 독립적**이다. y_{11}을 계산하는 데 y_{23}이 필요하지 않다. 따라서 여러 개를 **동시에(병렬로)** 계산할 수 있다. 행렬곱 XW 한 번은 바로 이런 독립적인 곱셈과 덧셈을 한꺼번에 모아 놓은 것이다.

- **NumPy**: `X @ W`는 Python의 for loop가 아니라, CPU에서 행렬 연산에 최적화된 수치 계산 라이브러리(BLAS)를 호출하여 여러 연산을 동시에(CPU의 여러 core와 벡터 연산 명령) 처리한다. 이렇게 반복문을 행렬 연산으로 바꾸는 것을 **벡터화(vectorization)**라 한다.
- **GPU**: 원래 화면의 수많은 픽셀을 동시에 계산하기 위해 만들어진 장치로, CPU보다 훨씬 많은 수천 개의 단순한 연산 장치를 가지고 있어 **같은 종류의 독립적인 계산을 대량으로 동시에** 하는 데 특화되어 있다. 신경망의 학습은 대부분 큰 행렬곱이므로 GPU에서 매우 빨라진다. 딥러닝에 GPU가 필요한 이유이다.
- **PyTorch의 `torch.tensor`**: NumPy 배열과 비슷한 다차원 배열이지만, `.to("cuda")`로 GPU에 올리면 `X @ W` 같은 연산을 GPU에서 병렬로 실행한다. (여기에 역전파를 자동으로 계산하는 autograd 기능이 더해진다.)

직접 확인해 보기: `loop_vs_matmul.py`

데이터 $h = 100$개, $l = 784$, $m = 100$인 affine 계층의 순전파를 (1) Python의 3중 for loop로, (2) `X @ W + B` 한 줄로 계산하여 시간을 비교한다.

```python
def forward_loop(X, W, B):
    A = np.zeros((X.shape[0], W.shape[1]))
    for k in range(X.shape[0]):            # one sample at a time
        for j in range(W.shape[1]):        # one output node at a time
            s = 0.0
            for i in range(X.shape[1]):    # a_kj = sum_i x_ki w_ij + b_j
                s += X[k, i] * W[i, j]
            A[k, j] = s + B[0, j]
    return A

def forward_matmul(X, W, B):
    return X @ W + B                       # all samples, all nodes at once
```

두 결과는 같지만, 강의 준비에 사용한 노트북 PC에서는 for loop가 약 1.8초, `X @ W + B`가 약 0.3ms로 **수천 배** 차이가 났다 (정확한 값은 컴퓨터마다 다르다). 미니배치를 행렬로 쌓아 한꺼번에 계산하는 이유가 여기에 있다.

이렇게 행렬 연산이 빠르기 때문에 데이터 h개를 처리하는 시간은 데이터 1개를 처리하는 시간의 h배보다 훨씬 적게 든다. 미니배치 크기 h를 너무 작게 잡으면 이 병렬성을 살리지 못하고, 너무 크게 잡으면 업데이트 횟수가 줄고 메모리가 부족해진다. GPU를 쓸 때 미니배치 크기를 32, 64, 128, 256 정도로 잡는 것은 이런 절충의 결과이다.

11.4 미니배치에 대한 역전파

데이터 X_k를 순전파했을 때의 손실을 l_k라 하고, 평균 손실을 $L = \frac{1}{h}\sum_k l_k$라 하자.

$$\frac{\partial L}{\partial X} := \begin{bmatrix} \frac{\partial L}{\partial X_1} \\ \vdots \\ \frac{\partial L}{\partial X_h} \end{bmatrix}_{h\times l}, \qquad \frac{\partial L}{\partial Y} := \begin{bmatrix} \frac{\partial L}{\partial Y_1} \\ \vdots \\ \frac{\partial L}{\partial Y_h} \end{bmatrix}_{h\times m} = \frac{1}{h}\begin{bmatrix} \frac{\partial l_1}{\partial Y_1} \\ \vdots \\ \frac{\partial l_h}{\partial Y_h} \end{bmatrix}$$

로 정의하자 (k번째 행이 k번째 데이터에 대한 기울기). 데이터 하나에 대한 결과 $\frac{\partial l_k}{\partial W} = X_k^\mathsf{T}\frac{\partial l_k}{\partial Y_k}$ (2절)를 이용하면

$$\frac{\partial L}{\partial W} = \frac{1}{h}\left(X_1^\mathsf{T}\frac{\partial l_1}{\partial Y_1} + \cdots + X_h^\mathsf{T}\frac{\partial l_h}{\partial Y_h}\right) = \frac{1}{h}\begin{bmatrix} x_{11}\frac{\partial l_1}{\partial y_{11}} + \cdots + x_{h1}\frac{\partial l_h}{\partial y_{h1}} & \cdots \\ \vdots & \ddots \end{bmatrix}$$

$$= \begin{bmatrix} x_{11} & x_{21} & \cdots & x_{h1} \\ \vdots & & & \vdots \\ x_{1l} & x_{2l} & \cdots & x_{hl} \end{bmatrix} \cdot \frac{1}{h}\begin{bmatrix} \frac{\partial l_1}{\partial y_{11}} & \cdots & \frac{\partial l_1}{\partial y_{1m}} \\ \vdots & & \vdots \\ \frac{\partial l_h}{\partial y_{h1}} & \cdots & \frac{\partial l_h}{\partial y_{hm}} \end{bmatrix} = X^\mathsf{T}\frac{\partial L}{\partial Y}$$

이다. 즉 h 개의 **외적의 합**이 행렬곱 $X^\mathsf{T}\frac{\partial L}{\partial Y}$ 한 번으로 계산된다. 마찬가지로

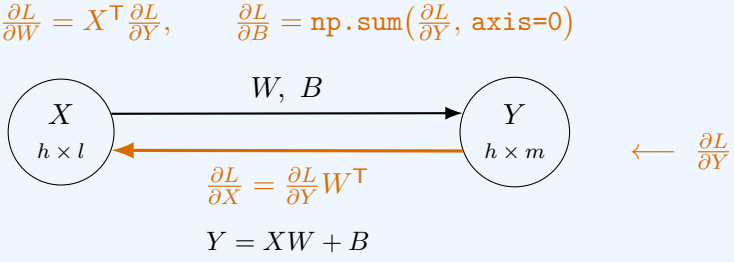
$$\frac{\partial L}{\partial B} = \frac{1}{h}\sum_{k=1}^{h}\frac{\partial l_k}{\partial Y_k} = \texttt{np.sum}\left(\frac{\partial L}{\partial Y}, \texttt{axis=0}\right), \qquad \frac{\partial L}{\partial X} = \begin{bmatrix} \frac{\partial L}{\partial Y_1}W^\mathsf{T} \\ \vdots \\ \frac{\partial L}{\partial Y_h}W^\mathsf{T} \end{bmatrix} = \frac{\partial L}{\partial Y}W^\mathsf{T}$$

이다. $\frac{\partial L}{\partial B}$ 는 $\frac{\partial L}{\partial Y}$ 의 h 개 행을 **세로 방향으로 더한** $1 \times m$ 행렬이다.

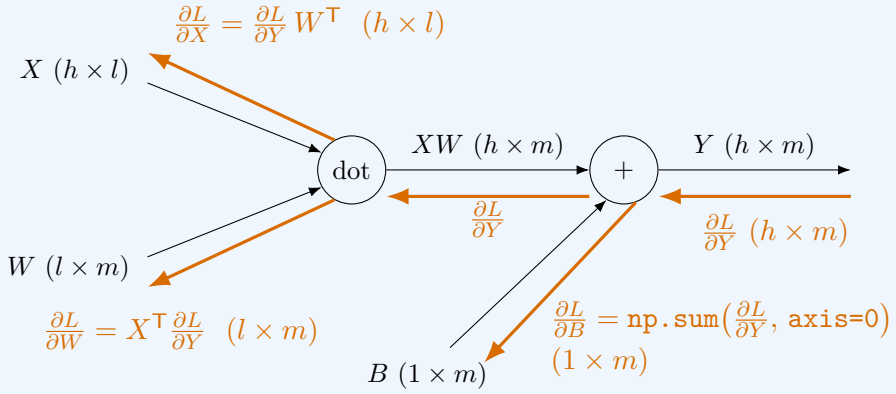
미니배치 affine 계층의 역전파

순전파 $Y_{h\times m} = X_{h\times l}W_{l\times m} + B_{1\times m}$ (B 는 broadcast)에 대하여

$$\left(\frac{\partial L}{\partial X}\right)_{h\times l} = \left(\frac{\partial L}{\partial Y}\right)_{h\times m}(W^\mathsf{T})_{m\times l}, \qquad \left(\frac{\partial L}{\partial W}\right)_{l\times m} = (X^\mathsf{T})_{l\times h}\left(\frac{\partial L}{\partial Y}\right)_{h\times m},$$
$$\left(\frac{\partial L}{\partial B}\right)_{1\times m} = \texttt{np.sum}\left(\frac{\partial L}{\partial Y}, \texttt{axis=0}\right)$$



(1) 변수 노드 기준



(2) 연산 노드 기준

데이터 하나일 때와 **같은 식**이며, B의 기울기만 h 개 행을 더한다. parameter의 업데이트도 같다:

$$W \leftarrow W - \alpha X^\mathsf{T}\frac{\partial L}{\partial Y}, \qquad B \leftarrow B - \alpha\,\texttt{np.sum}\left(\frac{\partial L}{\partial Y}, \texttt{axis=0}\right).$$

axis=0과 axis=1

`np.sum(M, axis=0)`은 행렬 M의 **행들을 더한다** (세로 방향, 결과는 열마다 하나). `np.sum(M, axis=1)`은 각 행 **안의 원소들을 더한다** (가로 방향, 결과는 행마다 하나). 미니배치에서는 행이 데이터이므로, 데이터에 대한 합은 `axis=0`, 한 데이터 안의 class에 대한 합(softmax의 분모)은 `axis=1`이다.

평균은 한 번만 낸다

$\frac{1}{h}$은 손실 $L = \frac{1}{h}\sum_k l_k$에서 단 한 번 나온다. 코드에서는 이 $\frac{1}{h}$을 마지막 softmax–CE 계층의 역전파에서 한 번만 곱하여

$$\frac{\partial L}{\partial U} = \frac{1}{h}(Y - T) \qquad (Y, T:\ h \times n)$$

를 넘겨 주고, 모든 affine 계층에서는 h개 데이터의 기여를 **더하기만** 한다. 만약 각 affine 계층에서도 다시 h로 나누면, 앞쪽 층으로 갈수록 $\frac{1}{h^2}$, $\frac{1}{h^3}$이 곱해져 손실 L의 정확한 gradient가 아니게 된다. (지난주의 "bias의 gradient는 왜 합인가?"와 같은 이야기이다.)

11.5 Epoch, iteration, 미니배치 크기

- **iteration**: 미니배치 하나로 순전파–역전파–업데이트를 한 번 하는 것
- **epoch**: 전체 훈련 데이터를 한 번씩 모두 사용하는 것

MNIST의 훈련 데이터 $N = 60000$개를 미니배치 크기 $h = 100$으로 학습하면

$$1\ \text{epoch} = 60000/100 = 600\ \text{iteration}$$

이다. 매 epoch마다 데이터의 순서를 무작위로 섞은 뒤 앞에서부터 h개씩 잘라 쓰면, 한 epoch 동안 모든 데이터를 정확히 한 번씩 사용한다. 순서를 섞는 데에는 `np.random.permutation`을 쓴다.

12 3교시 실습: NumPy로 MNIST 손글씨 숫자 분류

이번 실습에서도 PyTorch의 autograd를 사용하지 않는다. 지금까지 유도한 계층들을 **클래스**로 만들어 조립하고, MNIST 손글씨 숫자 60000개로 학습시킨 뒤 10000개의 test 데이터로 정확도를 확인한다. 전체 코드는 `mnist_lab.py`에 있으며, 실행하면 그림이 `figures` 폴더에 저장된다.

12.1 Step 1. 데이터 불러오기와 전처리

```
import os
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
os.makedirs("figures", exist_ok=True)

```

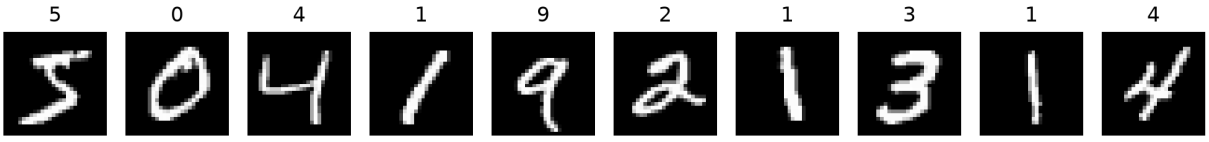
```python
def load_mnist():
    """Return (x_train, t_train), (x_test, t_test): images (N,28,28), labels (N,)."""
    try:
        from tensorflow.keras.datasets import mnist   # if TensorFlow is installed
        return mnist.load_data()
    except ImportError:
        from sklearn.datasets import fetch_openml     # otherwise download from OpenML
        X, y = fetch_openml("mnist_784", version=1, as_frame=False, return_X_y=True)
        X = X.reshape(-1, 28, 28).astype(np.uint8)
        y = y.astype(np.int64)
        return (X[:60000], y[:60000]), (X[60000:], y[60000:])

(x_train_o, t_train_o), (x_test_o, t_test_o) = load_mnist()

# each image (28, 28) -> one row vector (1, 784);  pixel values 0..255 -> 0..1
x_train = x_train_o.reshape(x_train_o.shape[0], -1) / 255.0   # (60000, 784)
x_test  = x_test_o.reshape(x_test_o.shape[0], -1) / 255.0     # (10000, 784)

# label k -> one-hot vector (row k of the 10x10 identity matrix)
t_train = np.eye(10)[t_train_o]   # (60000, 10)
t_test  = np.eye(10)[t_test_o]    # (10000, 10)
```

- 각 이미지 28×28을 한 줄로 펼쳐 1×784 행벡터로 만든다. 데이터 60000개가 행으로 쌓여 `x_train`의 shape은 `(60000, 784)`이다.
- 픽셀 값 $0 \sim 255$를 255로 나누어 $0 \sim 1$로 **정규화**한다. 입력이 너무 크면 XW가 커져 학습률을 아주 작게 잡아야 하고 학습이 불안정해진다.
- TensorFlow가 없으면 scikit-learn으로 MNIST를 내려받는다 (처음 한 번만 다운로드).



MNIST 훈련 데이터의 예 (위의 숫자는 정답 label)

12.2 Step 2. 계층(layer) 클래스

각 계층은 `forward`에서 역전파에 필요한 값을 저장해 두고, `backward`에서 upstream gradient를 받아 앞쪽 층으로 보낼 gradient를 돌려준다.

```python
class Affine:
    """A = X W + B  for a mini-batch X of shape (h, l)."""
    def __init__(self, W, B):
        self.W = W          # (l, m)
        self.B = B          # (1, m)
        self.X = None
        self.dW = None
        self.dB = None
```

```
9
10      def forward(self, X):
11          self.X = X
12          return X @ self.W + self.B          # (h,l)@(l,m) + (1,m) -> (h, m)
13
14      def backward(self, dA):
15          # dA = dL/dA (h, m). It already contains the 1/h of the mini-batch mean,
16          # so here we only SUM the contributions of the h samples.
17          self.dW = self.X.T @ dA                             # (l,h)@(h,m) -> (l, m)
18          self.dB = np.sum(dA, axis=0, keepdims=True)         # sum over rows -> (1, m)
19          return dA @ self.W.T                                # dL/dX : (h, l)
20
21
22  class Relu:
23      def __init__(self):
24          self.mask = None
25
26      def forward(self, A):
27          self.mask = (A > 0)                 # True where the node is active
28          return np.where(self.mask, A, 0.0)
29
30      def backward(self, dZ):
31          return np.where(self.mask, dZ, 0.0) # pass dZ only where A > 0
32
33
34  class Sigmoid:
35      def __init__(self):
36          self.Z = None
37
38      def forward(self, A):
39          self.Z = 1.0 / (1.0 + np.exp(-A))
40          return self.Z
41
42      def backward(self, dZ):
43          return dZ * self.Z * (1.0 - self.Z)
44
45
46  def softmax(U):
47      U = U - np.max(U, axis=1, keepdims=True)   # same result, avoids overflow of exp
48      expU = np.exp(U)
49      return expU / np.sum(expU, axis=1, keepdims=True)
50
51
52  class SoftmaxWithCrossEntropy:
53      def __init__(self):
54          self.Y = None
55          self.T = None
56
57      def forward(self, U, T):
58          self.T = T
59          self.Y = softmax(U)                                 # (h, n)
```

```
        h = U.shape[0]
        return -np.sum(T * np.log(self.Y + 1e-12)) / h       # mean over the mini-batch

    def backward(self):
        h = self.T.shape[0]
        return (self.Y - self.T) / h        # dL/dU : the ONLY place where we divide by h


class SoftmaxWithSquaredError:
    """Same softmax output, but loss = (1/h) * sum 0.5*(y - t)^2  (for comparison)."""
    def __init__(self):
        self.Y = None
        self.T = None

    def forward(self, U, T):
        self.T = T
        self.Y = softmax(U)
        h = U.shape[0]
        return 0.5 * np.sum((self.Y - self.T) ** 2) / h

    def backward(self):
        h = self.T.shape[0]
        R = self.Y - self.T                                  # dL/dY (times h)
        # multiply by the softmax Jacobian, row by row, without building it
        return self.Y * (R - np.sum(self.Y * R, axis=1, keepdims=True)) / h
```

코드와 수식의 대응은 다음과 같다.

계층	수식	코드
Affine	$\frac{\partial L}{\partial W} = X^{\mathsf{T}} \frac{\partial L}{\partial A}$	`X.T @ dA`
	$\frac{\partial L}{\partial B} = \sum_{\text{rows}} \frac{\partial L}{\partial A}$	`np.sum(dA, axis=0)`
	$\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A} W^{\mathsf{T}}$	`dA @ W.T`
ReLU	$\frac{\partial L}{\partial A} = \mathbf{1}[A > 0] \odot \frac{\partial L}{\partial Z}$	`np.where(mask, dZ, 0)`
Sigmoid	$\frac{\partial L}{\partial A} = Z \odot (1 - Z) \odot \frac{\partial L}{\partial Z}$	`dZ * Z * (1 - Z)`
Softmax–CE	$\frac{\partial L}{\partial U} = \frac{1}{h}(Y - T)$	`(Y - T) / h`
Softmax–SE	$\frac{\partial L}{\partial u_i} = \frac{1}{h}\, y_i \big(r_i - \sum_j y_j r_j\big),\ R = Y - T$	`Y * (R - np.sum(Y * R, axis=1, keepdims=True)) / h`

12.3 Step 3. 2층 신경망 클래스

```
class TwoLayerNet:
    def __init__(self, input_size, hidden_size, output_size, weight_init_std=0.01,
                 loss="ce"):
        self.params = {
            "W1": weight_init_std * np.random.randn(input_size, hidden_size),
            "B1": np.zeros((1, hidden_size)),
            "W2": weight_init_std * np.random.randn(hidden_size, output_size),
```

```
            "B2": np.zeros((1, output_size)),
        }
        self.affine1 = Affine(self.params["W1"], self.params["B1"])
        self.act     = Relu()                    # or Sigmoid()
        self.affine2 = Affine(self.params["W2"], self.params["B2"])
        if loss == "ce":
            self.last = SoftmaxWithCrossEntropy()
        else:                                    # loss == "se"
            self.last = SoftmaxWithSquaredError()

    def predict(self, X):                        # returns the scores U (h, n)
        A = self.affine1.forward(X)
        Z = self.act.forward(A)
        U = self.affine2.forward(Z)
        return U

    def loss(self, X, T):
        U = self.predict(X)
        return self.last.forward(U, T)

    def backward(self):                          # call right after loss()
        dU = self.last.backward()                # (h, n)
        dZ = self.affine2.backward(dU)           # (h, m)
        dA = self.act.backward(dZ)               # (h, m)
        dX = self.affine1.backward(dA)           # (h, l)
        return {"W1": self.affine1.dW, "B1": self.affine1.dB,
                "W2": self.affine2.dW, "B2": self.affine2.dB}

    def accuracy(self, X, T):
        U = self.predict(X)
        return np.mean(np.argmax(U, axis=1) == np.argmax(T, axis=1))
```

- `backward`는 10절의 역전파 순서를 그대로 코드로 옮긴 것이다.
- 예측 class는 확률이 가장 큰 class, 즉 `np.argmax`이다. softmax는 크기 순서를 바꾸지 않으므로 (6절의 성질 1)) 점수 U의 `argmax`와 같다.
- `Affine` 객체는 `params`의 배열을 **같은 메모리로** 참조한다. 따라서 업데이트는 `params[key] -= ...`처럼 제자리(in place)에서 해야 계층에도 반영된다.

12.4 Step 4. 기울기 검사 (gradient check)

역전파 코드가 맞는지는 **수치미분**과 비교하여 확인할 수 있다. 작은 network에서 각 parameter를 $\pm\varepsilon$만큼 바꿔 손실의 변화로 미분을 근사한다:

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon}.$$

```
def numerical_gradient(f, P, eps=1e-5):
    """Central difference of the scalar function f() w.r.t. every entry of array P."""
```

```
    grad = np.zeros_like(P)
    for idx in np.ndindex(P.shape):
        old = P[idx]
        P[idx] = old + eps
        f_plus = f()
        P[idx] = old - eps
        f_minus = f()
        P[idx] = old
        grad[idx] = (f_plus - f_minus) / (2 * eps)
    return grad

X_small = np.random.randn(6, 4)
T_small = np.eye(3)[np.random.randint(0, 3, size=6)]

for loss_type in ["ce", "se"]:
    small = TwoLayerNet(input_size=4, hidden_size=5, output_size=3,
                        weight_init_std=1.0, loss=loss_type)
    small.loss(X_small, T_small)
    grads = small.backward()
    for key in small.params:
        num = numerical_gradient(lambda: small.loss(X_small, T_small), small.params[key
    ])
        print(f"[{loss_type}] {key}: max |backprop - numerical| = "
              f"{np.max(np.abs(grads[key] - num)):.2e}")
```

차이가 10^{-8} 이하로 아주 작으면 역전파가 올바르게 구현된 것이다. (수치미분은 parameter 하나마다 순전파를 두 번 해야 하므로 느리다. 그래서 실제 학습에는 역전파를 쓰고, 수치미분은 **검증**에만 쓴다.)

12.5 Step 5. 미니배치 학습

```
network = TwoLayerNet(input_size=784, hidden_size=100, output_size=10)

epochs        = 10
batch_size    = 100
learning_rate = 0.1

N = x_train.shape[0]
iter_per_epoch = N // batch_size          # 600 iterations = 1 epoch

loss_history   = []
train_acc_list = []
test_acc_list  = []

for epoch in range(epochs):
    perm = np.random.permutation(N)       # shuffle once per epoch
    for it in range(iter_per_epoch):
        idx = perm[it * batch_size:(it + 1) * batch_size]
        X_batch, T_batch = x_train[idx], t_train[idx]

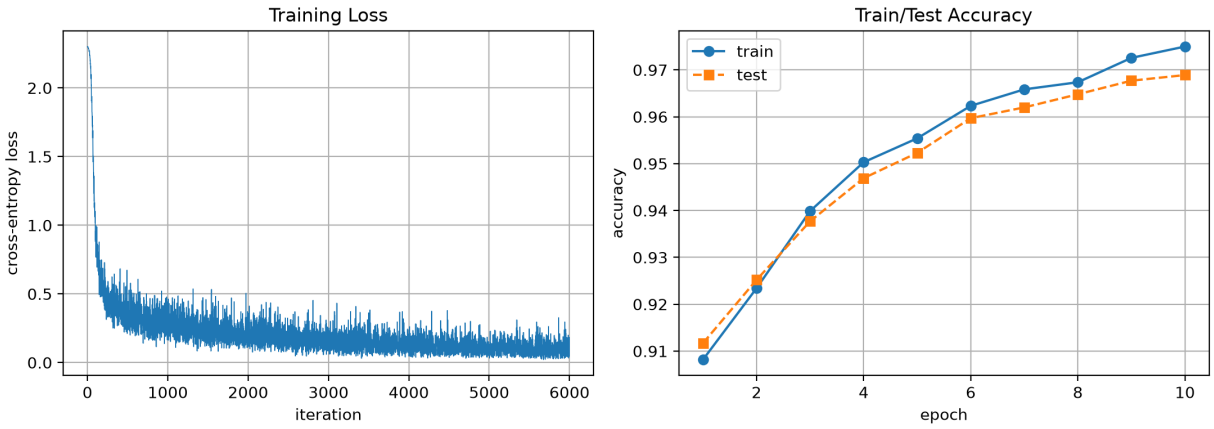
```

```
        loss  = network.loss(X_batch, T_batch)   # forward
        grads = network.backward()               # backward
        for key in network.params:               # gradient descent (in place)
            network.params[key] -= learning_rate * grads[key]

        loss_history.append(loss)

    train_acc = network.accuracy(x_train, t_train)
    test_acc  = network.accuracy(x_test, t_test)
    train_acc_list.append(train_acc)
    test_acc_list.append(test_acc)
    print(f"epoch {epoch + 1:2d}: loss={loss:.4f}, "
          f"train acc={train_acc:.4f}, test acc={test_acc:.4f}")
```

학습이 끝나면 손실과 정확도의 변화를 그려 `figures/mnist_loss_acc.png`로 저장한다 (그림 그리는 코드는 `mnist_lab.py` 참조).



iteration에 따른 미니배치 손실(왼쪽)과 epoch에 따른 train/test 정확도(오른쪽)

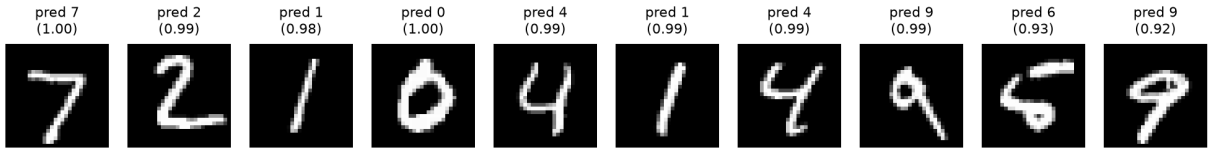
관찰해 보자

- 미니배치 손실 그래프는 왜 매끄럽지 않고 들쭉날쭉한가? (매 iteration마다 **다른** 100개의 데이터로 손실을 계산하기 때문이다.)
- train 정확도와 test 정확도의 차이는 어느 정도인가? test 데이터는 학습에 한 번도 쓰이지 않았다는 점을 기억하자.
- `learning_rate`를 0.01, 0.5로, `batch_size`를 10, 1000으로 바꾸면 학습 속도와 손실 그래프가 어떻게 달라지는가?
- `Relu()`를 `Sigmoid()`로 바꾸면 어떻게 되는가?
- `TwoLayerNet(..., loss="se")`로 제곱 오차를 쓰면 처음 몇 epoch의 정확도가 어떻게 달라지는가? (9절)

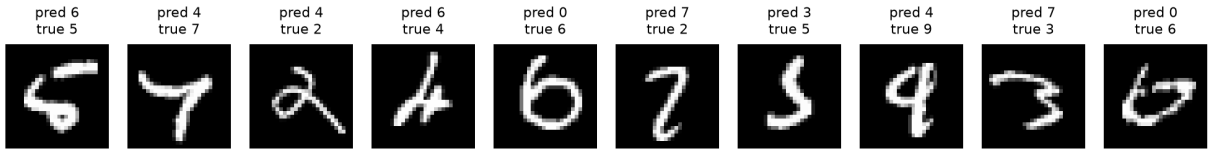
12.6 Step 6. 예측 결과 확인

```
U_test = network.predict(x_test)
Y_test = softmax(U_test)
pred   = np.argmax(Y_test, axis=1)

wrong = np.where(pred != t_test_o)[0]
print("number of wrong predictions:", len(wrong), "/", len(t_test_o))
```



test 데이터에 대한 예측 class와 그 확률 y_k

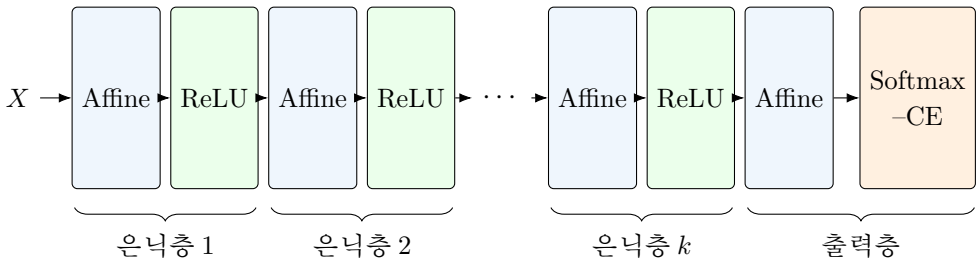


틀리게 예측한 test 데이터의 예 (pred: 예측, true: 정답)

틀린 예들을 보면 사람이 보기에도 헷갈리는 숫자가 많다. network는 각 숫자를 784개 픽셀 값의 함수로만 보고 있으며, 픽셀 사이의 공간적 구조(어떤 픽셀이 서로 이웃인지)는 전혀 사용하지 않는다는 점도 생각해 보자.

12.7 Step 7. 층을 깊이 쌓기: 완전 연결 심층신경망 (`mnist_mlp.py`)

지금까지 만든 network는 affine 계층과 활성화 계층을 한 번 거친 뒤 출력층으로 가는 2층 신경망이었다. 그런데 계층을 클래스로 만들어 두었으므로, **affine 계층과 활성화 계층의 조합을 원하는 만큼 반복해서 쌓기만** 하면 은닉층이 여러 개인 network를 바로 만들 수 있다.



각 affine 계층에서는 앞 층의 **모든** node가 다음 층의 **모든** node와 가중치로 연결되어 있다. 이렇게 affine 계층과 활성화 계층을 반복해 쌓은 network를 **완전 연결 신경망(fully connected neural network)** 또는 **다층 퍼셉트론(multi-layer perceptron, MLP)**이라 하고, 은닉층이 여러 개이면 **완전 연결 심층신경망**이라 한다.

깊이와 넓이로 표현력을 키운다

지난주에 hidden neuron을 늘리면 더 복잡한 함수를 근사할 수 있음을 보았다 (universal approximation). 마찬가지로

- 은닉층의 **넓이**(node 수)를 늘리거나,
- 은닉층의 **깊이**(층 수)를 늘려 앞 층이 만든 특징을 다시 조합하게 하면,

network가 표현할 수 있는 함수가 풍부해지고 분류 성능이 좋아질 수 있다. network 구조를 바꾸는 것만으로도, 즉 **깊이와 넓이의 조정만으로도** 성능을 끌어올릴 수 있다. 단, parameter가 많아질수록 훈련 데이터에만 지나치게 맞추는 **과적합**의 위험도 커진다.

새 계층을 만들 필요는 없다. `mnist_mlp.py`의 `MultiLayerNet`은 `mnist_lab.py`와 **똑같은** `Affine`, `Relu`, `Sigmoid`, softmax 계층을 리스트에 차례로 담아 쌓을 뿐이다.

```
class MultiLayerNet:
    def __init__(self, input_size, hidden_sizes, output_size,
                 activation="relu", loss="ce", weight_init_std=0.05):
        sizes = [input_size] + list(hidden_sizes) + [output_size]   # e.g. [784, 256, 128, 10]
        self.params = {}
        self.layers = []
        for i in range(len(sizes) - 1):
            n_in, n_out = sizes[i], sizes[i + 1]
            W = weight_init_std * np.random.randn(n_in, n_out)   # (input nodes, output nodes)
            B = np.zeros((1, n_out))
            self.params[f"W{i + 1}"] = W
            self.params[f"B{i + 1}"] = B
            self.layers.append(Affine(W, B))
            if i < len(sizes) - 2:                       # no activation after the last Affine
                self.layers.append(Relu() if activation == "relu" else Sigmoid())
        self.last = SoftmaxWithCrossEntropy() if loss == "ce" else SoftmaxWithSquaredError()

    def predict(self, X):                        # scores U
        for layer in self.layers:
            X = layer.forward(X)
        return X

    def loss(self, X, T):
        return self.last.forward(self.predict(X), T)

    def backward(self):                          # call right after loss()
        d = self.last.backward()
        for layer in reversed(self.layers):    # the same rule, layer after layer
            d = layer.backward(d)
        grads, k = {}, 1
        for layer in self.layers:
            if isinstance(layer, Affine):
```

```
                grads[f"W{k}"], grads[f"B{k}"] = layer.dW, layer.dB
                k += 1
        return grads
```

- `hidden_sizes`는 은닉층들의 넓이를 담은 리스트이다. 예를 들어

```
net = MultiLayerNet(784, [256, 128], 10)
```

  은 넓이가 256, 128인 은닉층 두 개를 가진 **3층** 신경망이다.

- 순전파는 계층 목록을 앞에서부터, 역전파는 **거꾸로** 돌며 각 계층의 `backward`를 호출한다. 층이 몇 개이든 이번 주에 유도한 규칙을 반복할 뿐이다.
- 가중치는 모두 표준편차 `weight_init_std`= 0.05인 정규분포로 초기화한다. 층이 깊어질 때 가중치를 어떻게 초기화해야 학습이 잘 되는지는 다음 주에 다룬다.

훈련 / 검증 / test 데이터

여러 network 중 어느 것이 좋은지를 고를 때 test 데이터를 보면 안 된다. test 정확도를 보면서 설정을 고르면, test 데이터에 맞춰 설정을 고른 셈이 되어 처음 보는 데이터에 대한 성능을 과대평가하게 된다. 그래서 `mnist_mlp.py`는 훈련 데이터 60000개 중 5000개를 떼어 **검증(validation) 데이터**로 두고,

1. 나머지 55000개로 학습하고,
2. 여러 설정을 **검증 정확도**로 비교하여 가장 좋은 것을 고른 뒤,
3. 고른 network의 성능을 test 데이터로 **단 한 번** 평가한다.

실험하는 방법

실험할 설정은 `mnist_mlp.py`의 `experiments` 리스트에 한 줄씩 적는다.

```
# Edit this list for Assignment 2. Each line is one experiment:
#   (name, hidden layer widths, activation, loss, learning rate, batch size)
# Change ONE thing at a time and compare the validation accuracy.
experiments = [
    ("A: [100]",      [100],      "relu", "ce", 0.1, 100),   # the network of the lab (
    Step 5)
    ("B: [100, 100]", [100, 100], "relu", "ce", 0.1, 100),   # one more hidden layer
]
epochs = 10
```

실행하면 각 설정의 epoch별 train/검증 정확도가 출력되고, `figures` 폴더에 그림 두 개가 저장된다.

- `mlp_compare.png`: 설정들의 epoch별 정확도 비교 그래프
- `mlp_confusion.png`: 검증 정확도가 가장 높은 network의 test 혼동행렬

혼동행렬(confusion matrix)의 i행 j열은 "정답이 i인데 j로 예측한 test 이미지의 개수"이며, 대각선이 맞힌 개수이다.

과제 2를 시작하며 생각해 볼 질문

- 은닉층을 넓게 하는 것과 깊게 쌓는 것 중 어느 쪽이 더 효과적인가? parameter 수가 비슷하도록 맞춰서 비교해 보라 (`num_params()`).
- train 정확도와 검증 정확도의 차이는 network가 커질수록 어떻게 변하는가?
- 학습률과 미니배치 크기를 바꾸면 손실 그래프와 정확도는 어떻게 달라지는가?
- ReLU 대신 sigmoid를, cross entropy 대신 제곱 오차를 쓰면 어떻게 되는가?
- 혼동행렬에서 가장 많이 헷갈린 숫자 쌍은 무엇인가?

13 이번 주의 핵심 정리

핵심 구조

다차원 다중분류 network:

$$\boxed{A = XW^{(1)} + B^{(1)}, \quad Z = h(A), \quad U = ZW^{(2)} + B^{(2)}, \quad Y = \mathrm{softmax}(U)}$$

$$\boxed{l = -\sum_i t_i \log y_i}$$

Affine 계층의 역전파 (한 데이터든 미니배치든 같은 식):

$$\boxed{\frac{\partial L}{\partial X} = \frac{\partial L}{\partial A} W^{\mathsf{T}}, \qquad \frac{\partial L}{\partial W} = X^{\mathsf{T}} \frac{\partial L}{\partial A}, \qquad \frac{\partial L}{\partial B} = \texttt{np.sum}\left(\frac{\partial L}{\partial A}, \texttt{axis=0}\right)}$$

활성화 계층의 역전파는 성분별 곱:

$$\boxed{\frac{\partial L}{\partial A} = h'(A) \odot \frac{\partial L}{\partial Z}} \qquad (\text{ReLU: } \mathbf{1}[A > 0], \ \text{sigmoid: } Z \odot (1 - Z))$$

Softmax–cross entropy:

$$\boxed{\frac{\partial l}{\partial U} = Y - t} \qquad (\text{미니배치 평균이면 } \tfrac{1}{h}(Y - T))$$

미니배치 학습:

$$\boxed{\text{평균은 손실에서 한 번만, 데이터에 대한 합은 행렬곱과 axis=0의 합}}$$

14 연습문제

1. MNIST network에서 $l = 784$, $m = 100$, $n = 10$, 미니배치 크기 $h = 100$이다. X, $W^{(1)}$, $B^{(1)}$, A, Z, $W^{(2)}$, $B^{(2)}$, U, Y, T의 shape을 쓰고, 학습해야 하는 parameter의 총 개수를 구하라.

2. $U = (1, 2, 3)$일 때 $Y = \mathrm{softmax}(U)$를 소수 셋째 자리까지 구하라. 또 임의의 상수 c에 대하여 $\mathrm{softmax}(U + c) = \mathrm{softmax}(U)$ 임을 보이라.

3. 정답이 $t = (0, 0, 1)$이고 network의 출력이 $Y = (0.2,\ 0.3,\ 0.5)$일 때 cross entropy 손실 l과 $\frac{\partial l}{\partial U}$를 구하라. 경사하강법은 세 점수 u_1, u_2, u_3를 각각 어느 방향으로 움직이는가?

4. Affine 계층 $A = XW + B$에서

$$X = (1,\ 2), \qquad W = \begin{pmatrix} 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix}, \qquad B = (0,\ 1,\ 0), \qquad \frac{\partial l}{\partial A} = (1,\ -1,\ 2)$$

   일 때 A, $\frac{\partial l}{\partial X}$, $\frac{\partial l}{\partial W}$, $\frac{\partial l}{\partial B}$를 구하라.

5. softmax의 Jacobian $J = \left(\frac{\partial y_j}{\partial u_i}\right)$가

$$J = \mathrm{diag}(y_1, \ldots, y_n) - Y^{\mathsf{T}} Y$$

   로 쓸 수 있음을 보이라. (Y는 $1 \times n$ 행벡터)

6. ReLU 계층의 역전파를 다음과 같이 구현한 코드가 있다.

```
def backward(self, dZ):
    return np.maximum(self.A, 0) * dZ
```

   무엇이 잘못되었는지 설명하고 올바르게 고쳐라.

7. 미니배치 학습에서 softmax–CE 계층이 `(Y - T) / h`를 돌려주는데, affine 계층의 `backward`에서도 `self.dW = self.X.T @ dA / h`처럼 다시 h로 나누었다고 하자. $\frac{\partial L}{\partial W^{(2)}}$와 $\frac{\partial L}{\partial W^{(1)}}$는 정확한 값의 몇 배가 되는가?

8. (도전) 손실함수를 제곱 손실 $l = \frac{1}{2}\sum_i (y_i - t_i)^2$으로 바꾸면

$$\frac{\partial l}{\partial u_i} = y_i\Big((y_i - t_i) - \sum_{j=1}^{n} y_j (y_j - t_j)\Big)$$

   임을 보이라 (9절). 또 $R = Y - T$라 할 때 이 식을 미니배치 행렬로 쓰면

```
Y * (R - np.sum(Y * R, axis=1, keepdims=True))
```

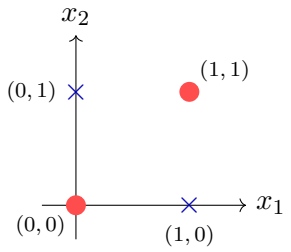
   가 됨을 확인하라.

15 과제

XOR 문제

XOR 문제는 인공지능과 신경망에서 비선형 분류 문제의 대표적인 예제로, 신경망의 학습 능력을 설명할 때 자주 사용되는 문제이다. XOR(배타적 논리합)은 **두 입력이 다를 때만 출력이 1**이 되는 이진 연산 문제로서 다음과 같이 표와 그림으로 설명할 수 있다.

입력 $X_i = (x_1, x_2)$	정답, 레이블	원핫벡터 $t_i = (t_1, t_2)$
$(0, 0)$	0	$(1, 0)$
$(1, 0)$	1	$(0, 1)$
$(0, 1)$	1	$(0, 1)$
$(1, 1)$	0	$(1, 0)$



위와 같이 두 입력 x_1, x_2가 다를 때에는 1을 출력하고, 같을 때에는 0을 출력하는 문제인데, 이것을 이차원 평면에 나타내면 네 개의 점을 두 개의 그룹으로 나누는 것이다. XOR 문제는 인공지능, 특히 신경망(neural networks) 연구의 역사에서 중요한 전환점을 나타내는 문제인데, 이 간단한 논리 연산이 AI 발전의 중단과 부흥을 모두 상징하는 사례로 여겨지는 이유는 다음과 같다.

1958년 로젠블랫이 퍼셉트론(perceptron)을 제안하면서 최초의 인공신경망의 개념이 탄생하였다. 퍼셉트론이란 은닉층이 없는 신경망으로 입력층에서 신호를 받아 활성화 함수를 거쳐 값을 출력하는 단순한 신경망을 말한다. 로젠블랫은 퍼셉트론을 학습시키는 알고리즘을 제시하였고 이로써 기계학습이 세상의 모든 문제를 해결할 수 있으리라는 기대감에 인공지능에 대한 연구는 큰 부흥기를 맞으며 천문학적인 연구 예산이 투입된다. 그러나 이후 단순히 퍼셉트론으로는 XOR 문제를 풀 수 없다는 것을 민스키와 페퍼트가 증명하면서, 인공지능의 첫 번째 겨울이 찾아왔다. 당시 민스키와 페퍼트를 비롯한 사람들은 XOR 문제를 여러 개의 퍼셉트론을 계층적으로 연결한 신경망, 즉 은닉층이 있는 다층 신경망(Multi-Layer Perceptron, MLP)으로 구현할 수 있을 것이란 것은 알고는 있었지만, 그러한 다층 신경망을 훈련하는 알고리즘은 존재하지 않을 것으로 여겼기 때문에 인공신경망에 대한 연구가 암흑기를 맞이한다. 이후 1986년에 David Rumelhart, Geoffrey Hinton, Ronald Williams가 다층 신경망을 훈련하는 알고리즘을 개발하였는데, 바로 이것이 경사하강법과 합성함수 미분법을 바탕으로 한 역전파(backpropagation) 알고리즘이다. 은닉층이 하나 이상일 때 신경망을 훈련하는 방법이 완성되면서 인공지능이 다시 부흥기로 들어서게 되었고, 수많은 은닉층을 갖는 인공신경망, 즉 심층신경망(Deep Neural Network, DNN)을 활용한 연구가 다시 부흥기를 맞이한다.

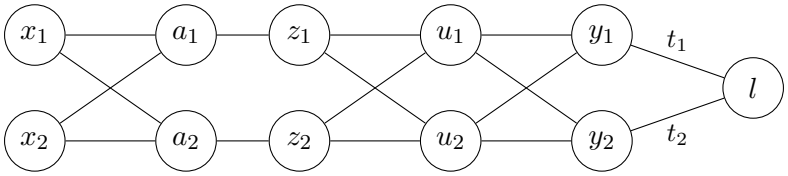
과제 1. XOR 문제의 온라인 학습을 손으로 계산하기

XOR 문제를 은닉층이 1개이고 출력층에 softmax를 사용하는 다음과 같은 신경망으로 학습시키는 과정을 연습해 보자.

다음과 같이 변수를 정의하자.

$$X = (x_1, x_2): \text{ 입력 데이터}, \qquad t = (t_1, t_2): \text{ 정답 레이블의 원핫 인코딩}$$

$$A = (a_1, a_2), \quad Z = (z_1, z_2), \quad U = (u_1, u_2), \quad Y = (y_1, y_2).$$



가중치와 편이를

$$W^{(1)} = \begin{pmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{pmatrix}, \quad B^{(1)} = \left(b_1^{(1)}, b_2^{(1)}\right), \quad W^{(2)} = \begin{pmatrix} w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} \end{pmatrix}, \quad B^{(2)} = \left(b_1^{(2)}, b_2^{(2)}\right)$$

라 하고, 신경망의 순전파를

$$A = XW^{(1)} + B^{(1)}, \quad Z = h(A), \quad U = ZW^{(2)} + B^{(2)}, \quad Y = \mathrm{softmax}(U), \quad l = -\sum_{i=1}^{2} t_i \log y_i$$

로 정의하자. 초기 가중치와 편이가

$$W_0^{(1)} = \begin{pmatrix} 2 & -1 \\ -2 & 2 \end{pmatrix}, \quad B_0^{(1)} = (1, 0), \quad W_0^{(2)} = \begin{pmatrix} -1 & 1 \\ -2 & 2 \end{pmatrix}, \quad B_0^{(2)} = (1, -1)$$

로 설정되었을 때, 데이터 $X_4 = (1, 1)$, $t_4 = (1, 0)$에 대한 온라인 학습을 진행하여 가중치를 업데이트하여라. 이때 $h(x) = \begin{cases} x, & x \geq 0 \\ 0, & x < 0 \end{cases}$로 ReLU 함수를 사용하고, 학습률은 1로 계산하고, 실습 코드의 계층 클래스를 활용하여 계산 결과를 확인하여라.

과제 2. MNIST 정확도 98% 도전

MNIST 숫자 분류 문제를 실습 코드 `mnist_mlp.py`(12.7절)를 활용하여 연습하여라. 은닉층의 수(깊이), 은닉층의 노드의 수(넓이), 활성화 함수의 종류, 손실함수의 종류, 학습률, 미니배치 크기, epoch 수 등을 조정하여 테스트 데이터에 대한 정확도를 98% 이상으로 훈련시켜라.

- 설정은 **검증 정확도**로 비교하여 고르고, test 정확도는 최종 선택한 network에 대해서만 보고하라.
- 한 번에 **한 가지 요인만** 바꾸어 실험하고, 시도한 설정과 그 결과(train/검증 정확도, 그래프)를 표로 정리하라.
- 어떤 요인이 정확도에 가장 큰 영향을 주었는지, 과적합의 징후가 있었는지 설명하라.

다음 주 예고

이번 주에는 은닉층이 1개인 network를 계층(layer) 단위로 구현하였다. 같은 `Affine`과 활성화 계층을 여러 번 쌓으면 은닉층이 여러 개인 **심층신경망**이 된다. 다음에는 층이 깊어질 때 학습이 잘 되도록 하는 방법 (최적화 기법, 가중치 초기화, 오버피팅, 드롭아웃 등)을 다룬다.